

ファミコンゲームの作り方

松原 拓也

Copyright © 2026 Takuya Matubara

All rights reserved.

はじめに

ファミコンのゲームを作り方を紹介する本です。2018年ごろ、「ねこたこ」というゲームに触発されて、スーパーファミコン用のカセットの基板を作ったのですが、それをきっかけに雑誌「日経ソフトウエア」さんからご提案を頂いて、2020年から2022年まで、レトロゲーム機のプログラミングの特集記事を書かせて頂きました。扱ったハードはメガドライブ、ファミコン、ゲームボーイ、PCエンジンです。「日経ソフトウエア」が休刊になってヒマになったので、ファミコン関連の記事を全てゼロから書き直しました。一部のプログラムは雑誌で作ったものを流用しています。こういう時代なので、念のため書いておくと、AIは一切使っていません。この本は、きちんとした「形」になって欲しいので、いつか印刷したいと思っています。誤植を見つけたら今のうちに教えてください。

松原 拓也

ご注意/免責
・本書で紹介している作品についての動作は保障していません。
・本書が原因で発生した事故については責任を負いかねます。
＊「ファミリーコンピュータ」は任天堂の商標です。
＊文中に登場する製品名はそのメーカーの商標です。

もくじ

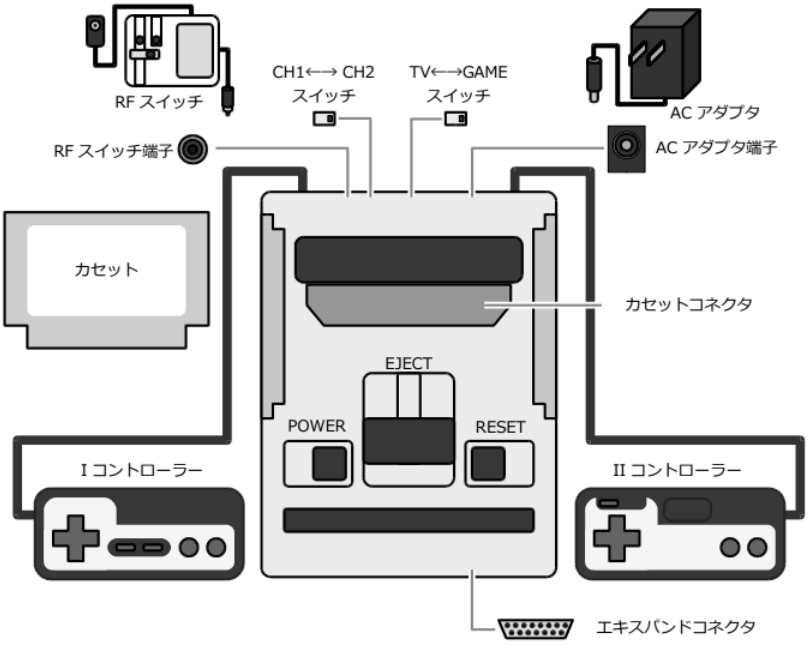
はじめに 3
第1章 ファミコンとは 6
1-1 基本的な情報 6
1-2 動く仕組み 9
1-3 メモリマップ 13
1-4 画面の仕組み 16
第2章 開発環境の準備 20
2-1 開発ツールを入手する 20
2-2 HELLO WORLD 24
2-3 タイル画像を描く 38
2-4 デモプログラム 42
第3章 BGを表示する 58
3-1 BGの仕様 58
3-2 BGのパレット 60
3-3 BGのスクロール 65
第4章 スプライトを表示する 67
4-1 スプライトの仕様 67
4-2 スプライトのパレット 71
4-3 スプライトの衝突 73
4-4 BGとの衝突 75
第5章 コントローラーの入力 78
5-1 一人で遊ぶ 80
5-2 二人で遊ぶ 81
5-3 四人で遊ぶ、その1 81
5-4 四人で遊ぶ、その2 82
第6章 サウンド 84
6-1 矩形波 85

6-2 三角波 90
6-3 ノイズ 90
6-4 DPCM 91
第7章 ファミコンゲームを作る 93
7-1 ジャンプゲーム 93
7-2 シューティングゲーム 101
第8章 実機で動かす 110
8-1 カセットの自作 112
8-2 ライターの自作 114
参考資料 116



第1章 ファミコンとは

1-1 基本的な情報

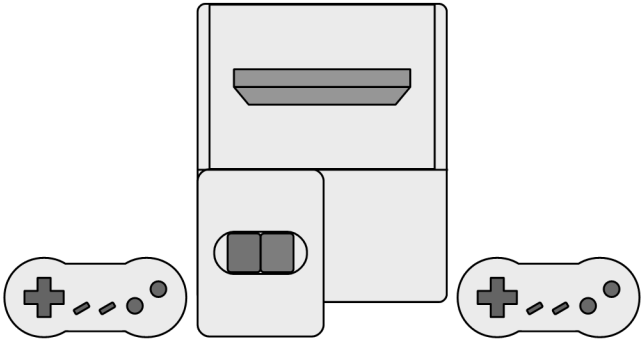


「ファミリーコンピュータ(ファミコン)」は任天堂が1983年7月に発売した家庭用ゲーム機です。発売当時の価格は1万4800円でした。ファミコンは優れた映像とサウンド、拡張性の高さ、豊富なソフトによって、一大ブームを巻き起こしました。任天堂の公式Webサイトによると、ファミコンとNES(海外版ファミコン)の累積販売台数は「6191万台」とのことです。

ファミコンは「カセット」を挿し込むことでソフトを交換します。「コントロー

ラー」は本体に2つ搭載されています。プレイヤー2のコントローラーにはマイクが搭載されています。「エキスパンドコネクタ」には周辺機器を接続することができます。ファミコンの電源は「ACアダプタ」から供給します（純正品はDCプラグがセンターマイナス。電圧はDC9Vです）。テレビに映すには「RFスイッチ」という信号の切り替え器を接続します。さらにテレビはチャンネルを「1」や「2」に切り替える必要がありました。1980年代前半はまだビデオ端子付きのテレビが普及していなかったため、ファミコンはアナログ放送の原理を使って映像と音を出力していました。今、売られているテレビはアナログ放送に対応していないため、そのままではファミコンを接続することができません。アナログ放送のチューナーを接続するか、ビデオ出力ができるようにファミコンを改造する必要があります。

《コラム》 ニューファミコン



初代ファミコンの発売から10年後の1993年に「AV仕様ファミコン（通称、ニューファミコン）」が発売されました。AV仕様ファミコンはビデオ出力端子が搭載されています。コントローラーはNESと同じコネクタを採用しています。

《コラム》 ファミリーベーシック

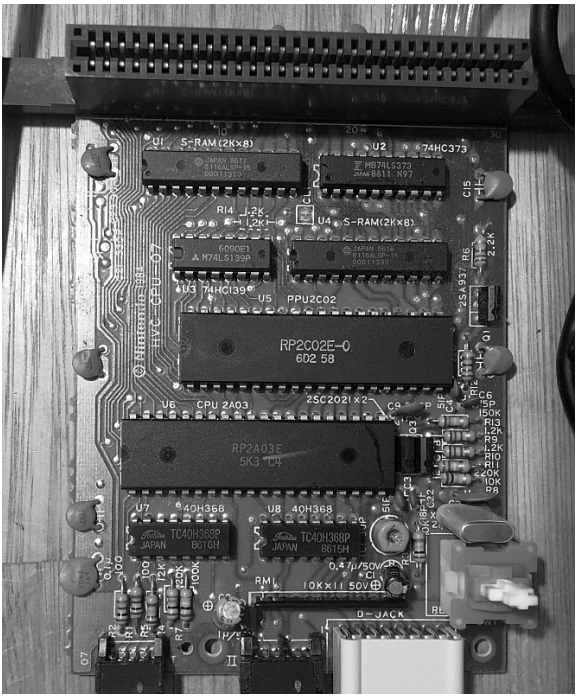


「ファミリーベーシック(1984年)」はファミコンでBASICのプログラムを作るカセットとキーボードのセットです。命令はPEEK、POKE、CALLに対応していて、マシン語も実行できます。別売のデータレコーダーでプログラムをテープに録音することができます。カセットに乾電池を入れると、バックアップRAMで内容を保持することができます。バックアップRAMの容量が2KBしかないことが難点です(V3は4KB)。ファミコンがパソコンに変身できるとアピールしていて商売上手です。

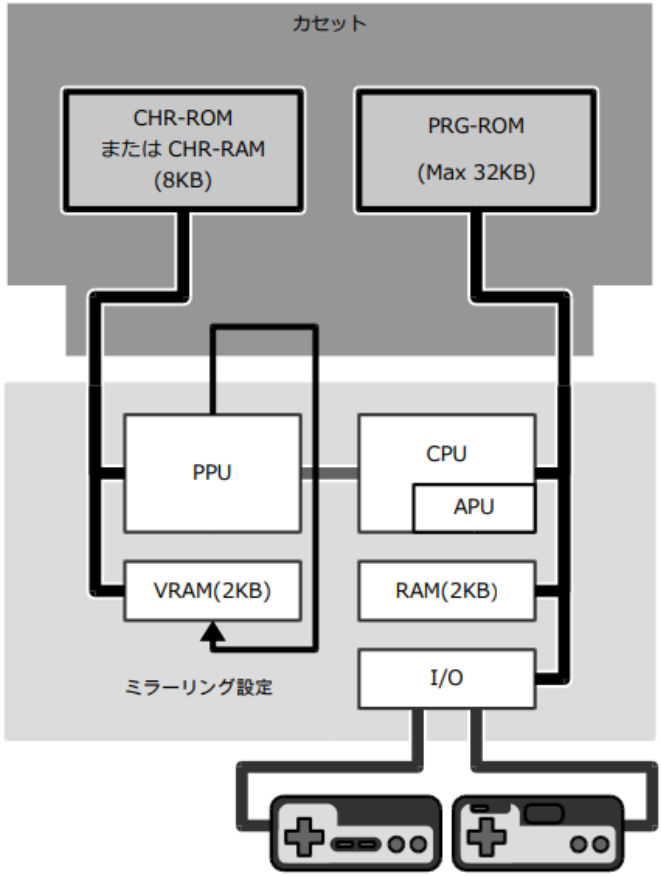
《コラム》 NESのプロテクト

海外版ファミコンは「Nintendo Entertainment System」。通称「NES」です。NESは1985年に米国で発売され、その後、世界中で発売されました。「NES」のカセットと本体には正規品だけを動かすチップとして、「CIC(Checking Integrated Circuit)」が搭載されています。CICの中身は知的財産として守られているため、勝手に複製することはできません。

1-2 動く仕組み



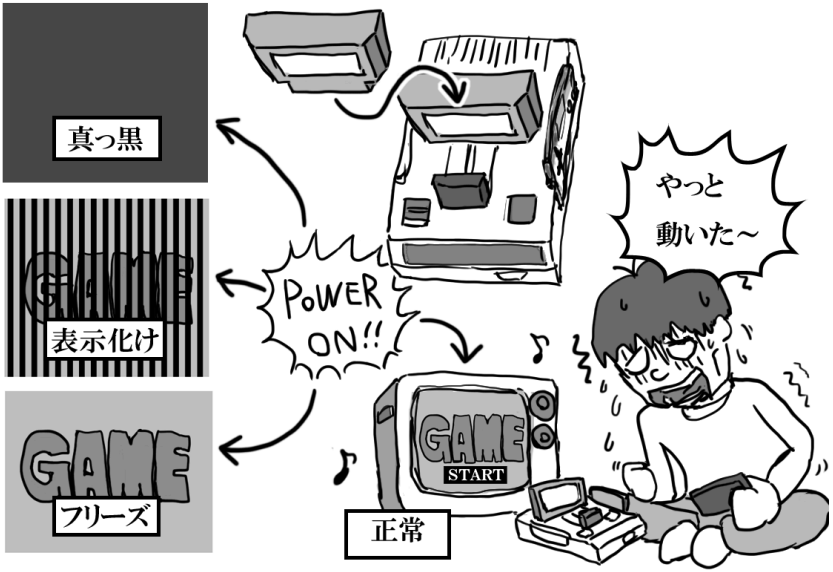
上の写真はファミコンの基板です。基板は製造時期によって細かい点が違いますが、性能は全て同じです。「RP2C02E」と書かれているのがリコー製の「PPU(Picture Processing Unit)」です。PPUは映像を出力するためのチップです。「RP2A03E」と書かれているのがリコー製の8ビットCPU(Central Processing Unit)です。CPUはプログラムを実行するための部品です。CPUの中にはAPU(Audio Processing Unit)も搭載されています。チップをゲームに特化させることで、ギリギリまでコストダウンしているのがファミコンの凄いところです。



ファミコンの部品の構成図がこちらです。カセットの中には「PRG-ROM」と「CHR-ROM」または「CHR-RAM」という部品が搭載されています。PRG-ROMはプログラムを記録したROMです。CHR-ROM/CHR-RAMは画像データを記録したROM/RAMです。各部品をつなぐ信号線の集まりのことを「バス」といいます。バスは「データバス」と「アドレスバス」で1組ですが、この図では1本の線で表現しています。アドレスはメモリ空間の「番地」のことです。カセットコネクタには「CPU用のバス」と「PPU用のバス」が2組あります。VRAMはCPU側の

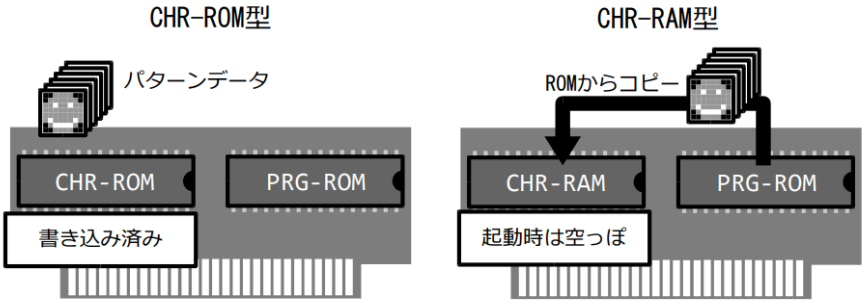
バスにつながっていません。そのため、CPUから直接VRAMを書き換えることができません。画面の表示を変えるにはPPUを制御しないといけません。

本体の値段を下げるためだと思いますが、カセットコネクタのバスを2組にするというのは、ちょっと無理のある設計に感じます。カセットコネクタの端子が多すぎて、接触不良が起こりやすくなっています。スーパーファミコンはCHR-RAMを本体に置き、カセットコネクタのバスを1組に改良しています。とはいえ、PPUのバスを外部に出したメリットは大きくて、カセットにVRAMを入れるような過激な機能拡張が可能となりました。



《コラム》ROMとRAM

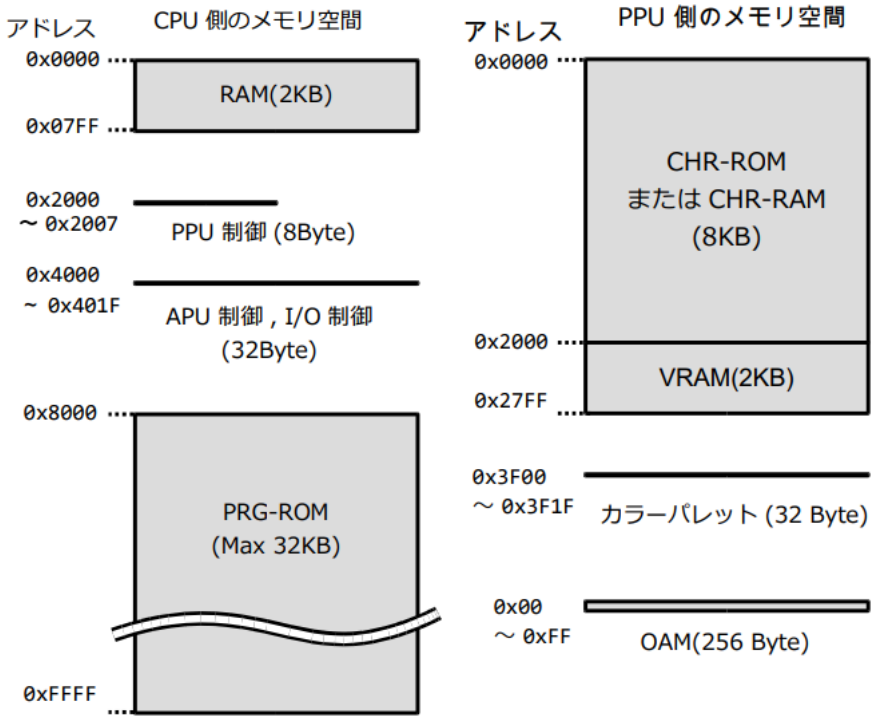
ROM(Read Only Memory)とは、読み込み専用のメモリのことです。電源を切っても内容が消えないため、不揮発性メモリと呼びます。ファミコンのカセットに使われているのはマスクROMです。マスクROMは工場で一度内容を書き込んだら、二度と書き換えることはできません。ROMの種類の中には紫外線で内容を消すことができるUVEPROMや、電気的に内容を消すことができるEEPROMがあります。EEPROMと似ていますが、ページ単位で内容を消すことができるフラッシュメモリもあります。RAM(Random Access Memory)は読み書きのできるメモリのことです。ROMと違って、電源を切ると内容が消えてしまいます。



カセットの種類のことをファミコンのエミュレータでは「マッパー」と呼びます。マッパーは260種類以上もあります。最もオーソドックスなマッパーは「マッパー0」です。プログラムを作る前にカセットの種類をあらかじめ決めておきましょう。今回はカセットを自作することを考えて「CHR-RAM型のマッパー0」という珍しい仕様を採用します。カセットの中には「PRG-ROM」と「CHR-ROM」または「CHR-RAM」が必要です。

CHR-RAMの場合、電源を入れた直後はRAMの中身が空っぽです。このため、PRG-ROMからCHR-RAMへデータをコピーする必要があります。余分な手間が増えてしまいますが、焼き込むROMは1つで済みます。

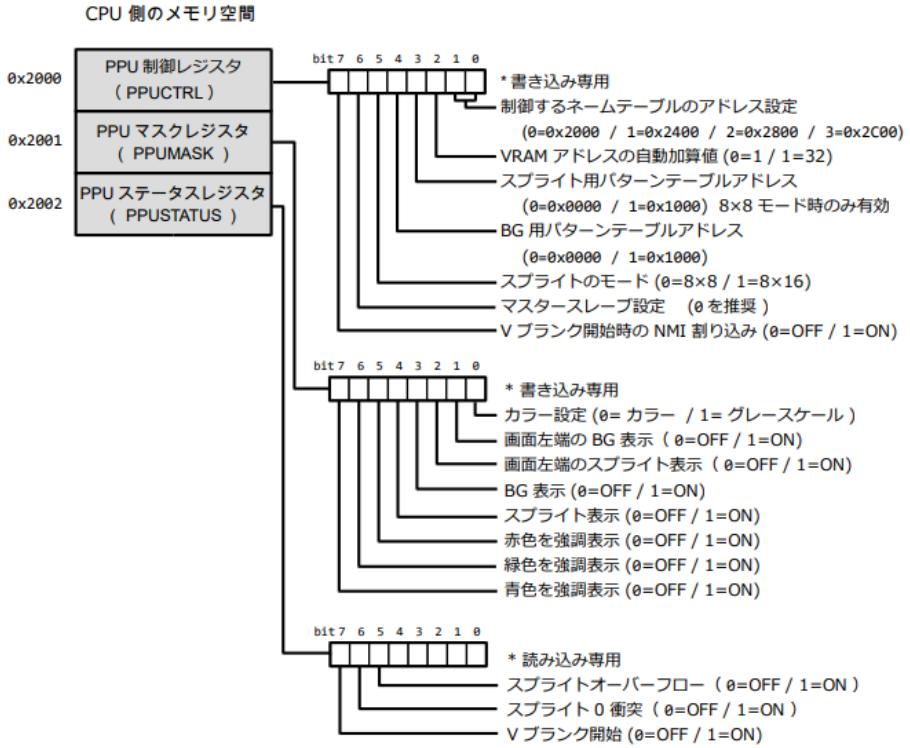
1-3 メモリマップ



メモリマップです。CPUとPPUでそれぞれ別々のメモリ空間があります。CPU側のメモリ空間は、0x0000～0x07FF番地がRAMです。0x2000～0x2007番地がPPU制御レジスタ、0x4000～0x401F番地がAPU制御レジスタとI/O制御レジスタです。0x8000～0xFFFF番地がPRG-ROMです。PRG-ROMにはプログラムを格納します。ここに載ってないアドレス

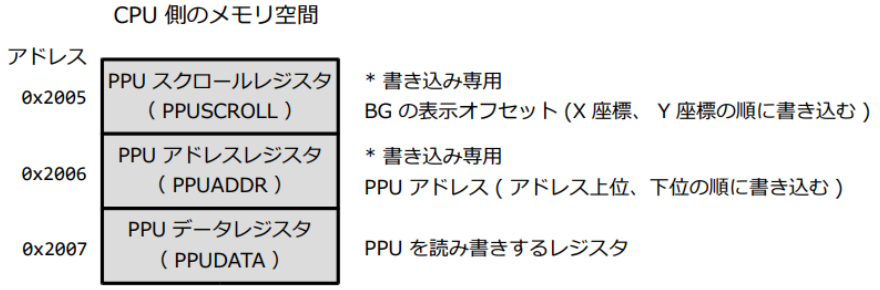
を使うカセットもあります。たとえば、バッテリーバックアップRAM（8KB）を搭載したカセットならば、0x6000～0x7FFF番地を使用します。

PPU側のメモリ空間は、0x0000～0x1FFF番地がCHR-ROMまたはCHR-RAMです。ここはタイル画像を格納する領域（パターンテーブル）です。0x2000～0x2FFF番地はVRAMです。VRAMにはBGの情報を格納します。0x3F00～0x3F1F番地はカラーパレットです。PPUにはOAMというスプライトデータを格納する領域があります。OAMは独自のアドレスを持っています。



上の図はCPUからPPUを制御するためのレジスタです。「PPU制御レジスタ（PPUCTRL）」はパターンテーブルの割り当てを設定したり、ネームテーブ

ルの制御対象などを設定したり、スプライトの表示モードを設定することができます。「PPUマスクレジスタ（PPUMASK）」は画面表示の有効/無効を設定することができます。珍しいところでは、画面の左端の8ピクセルを非表示/表示を設定することも可能です。「PPUステータスレジスタ（PPUSTATUS）」はVブランクの開始などを検出することができます。



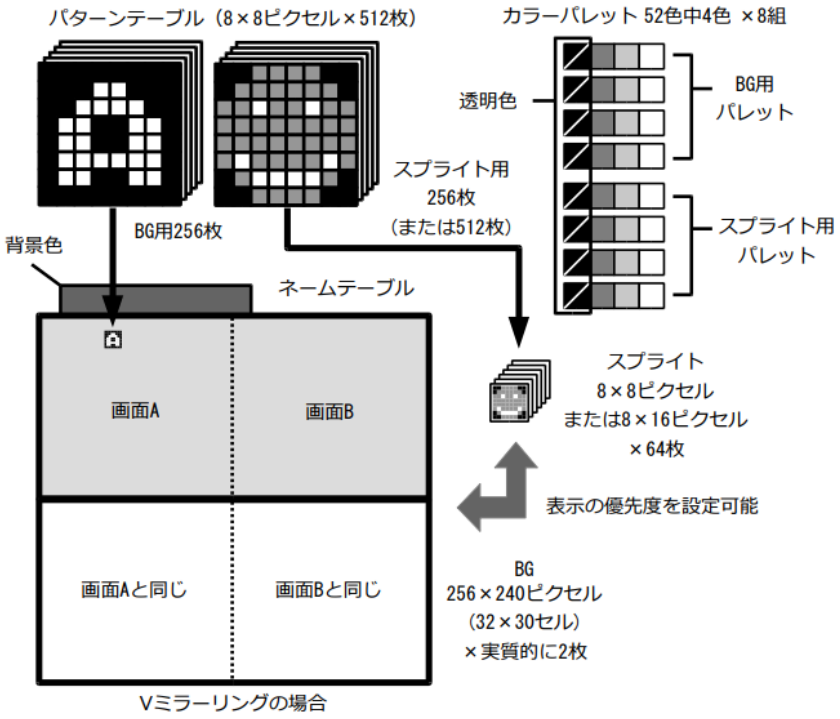
「PPUスクロールレジスタ（PPUSCROLL）」はBGのオフセット座標を設定することができます。BGのスクロール時に使います。「PPUアドレスレジスタ（PPUADDR）」と「PPUデータレジスタ（PPUDATA）」はCPUがPPUのメモリ空間を読み書きする時に使うレジスタです。データレジスタを読み書きすると、アドレスレジスタの値は自動的に増えていきます。このレジスタはVブランク中か表示の停止中に1バイトずつ読み書きしないといけません。

《コラム》 バンク切り替え

1985年発売の「スーパーマリオブラザーズ」の容量はPRG-ROMが32KB、CHR-ROMが8KB。合計で40KBです。発売当初は問題が無かったと思いますが、ゲームがどんどん大規模化していったため、容量が全く足りなくなってしまいました。この限界を超えるのが「バンク切り替え」です。バンク切り替えは

ロジックICや特殊チップを使って、一定の容量ごとにROMの中身を切り替える技術のことです。この技術の登場によって、カセットの容量が一気に増大しました。たとえば、1990年発売の「ドラゴンクエストIV」のPRG-ROMは512KB(4Mbit)です。1986年には「ファミリーコンピュータ ディスクシステム」が発売されて、「カセットの3倍の容量があるディスクカード」を誇っていたのですが、今度はカセットがそれをあっさりと追い抜いてしまいました。

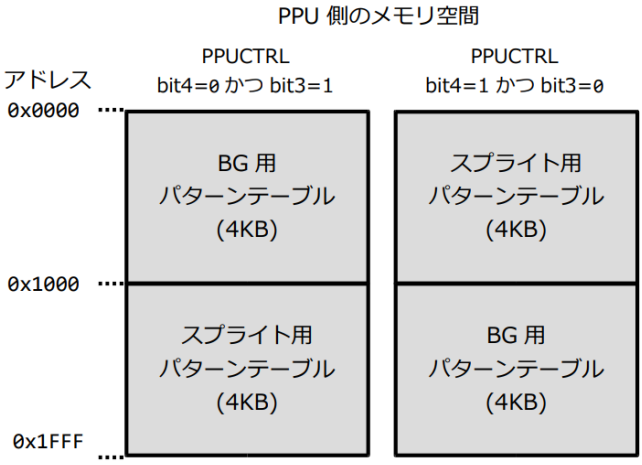
1-4 画面の仕組み



ファミコンは「BG」と「スプライト」を重ね合わせて画面を表示します。BGは背景や文字を表示する機能です。スプライトはゲームのキャラク

ターなどを表示する機能です。スプライトは公式では「オブジェクト」と呼びます。スプライトは左右反転や上下反転して表示することができます。BGの奥にスプライトを表示することもできます。「背景色」は常に一番奥に表示されます。

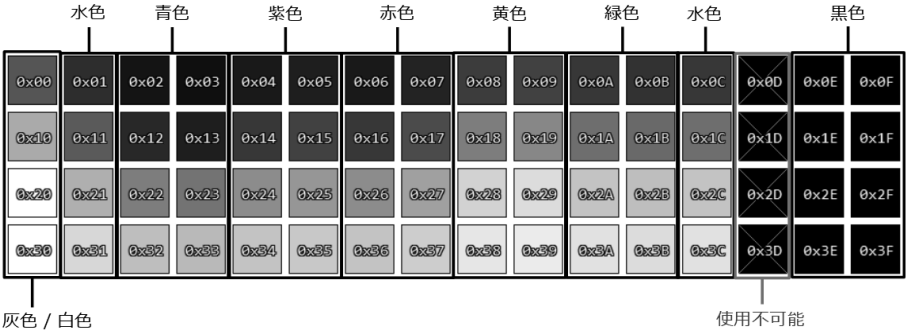
BGやスプライトの画像の素材を「パターン」と呼びますが、ここでは「タイル画像」と呼ぶことにします。タイル画像は「パターンテーブル」に格納します。タイル画像のサイズは1枚あたり8x8ピクセルです。



「パターンテーブル」はカセット内部のCHR-ROM/CHR-RAMに存在します。パターンテーブルの容量は8KBです。「PPU制御レジスタ(PPUCTRL)」で設定することで、BG用かスプライト用かを4KB単位で割り当てることができます。ややこしいのですが、スプライトが8×16モードの場合は8KBをまるごと使えるので、スプライト用の割り当ての設定は無視されます。

BGの表示内容はVRAMの中にある「ネームテーブル」に格納します。BGは2×2=4枚の画面が並んでいる構造をしています。このうち2枚の画面は隣の画面をコピーしているだけです。詳細は後の章で紹介します。

ファミコンの表示色



表示色は全部で52色あります。色は完全な青や赤などがなく、混ざり合っています。色は16進数のカラーコード(0x00～3F)で表現します。

・0x□0:灰色／白色

・0x□1:水色(シアン)

・0x□2～□3:青色

・0x□4～□5:紫色(マゼンタ)

・0x□6～□7:赤色

・0x□8～□9:黄色

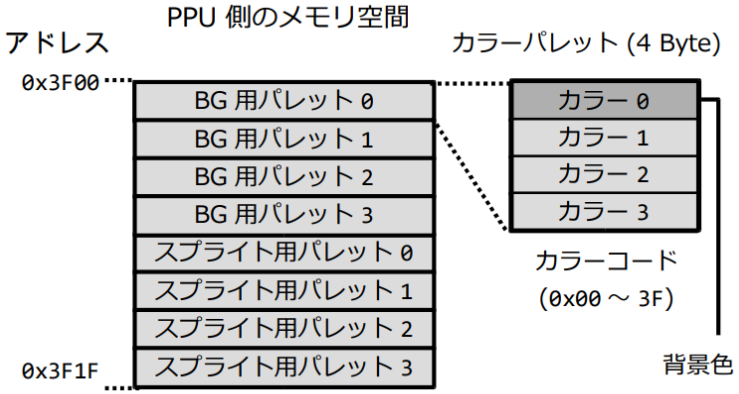
・0x□A～□B:緑色

・0x□C:水色(ティール)

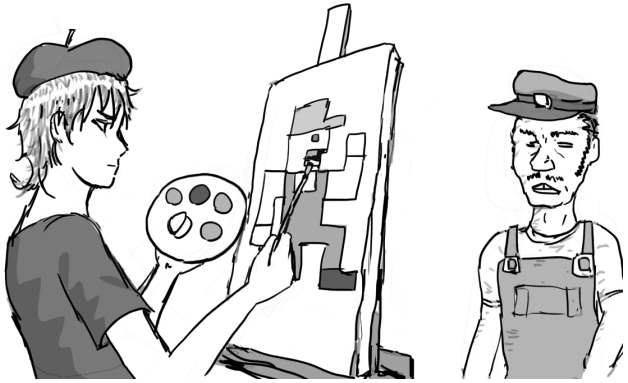
・0x□D:使用できない色

・0x□E～□F:黒色

カラーコードの下位1ケタは色の種類(0～F)です。上位1ケタは明るさ(0～3)です。



使用する色はPPU内にある「カラーパレット」で設定します。パレット1つで4色です。パレットはBG用が4つ、スプライト用が4つです。記憶領域は8パレット×4バイト=32バイトです。色はカラーコード(0x00～3F)で設定します。パレットに登録する色を「インデックスカラー」と呼ぶことにします。インデックスカラー0番は「透明色」として固定されているため、実際に設定できるのは1～3番だけです。ただし、「BG用パレット0」と「スプライト用パレット0」だけは例外で、インデックスカラー0番の色にカラーコードを書き込むと「背景色」と解釈されます。それ以外のパレットは0番の色にカラーコードを書き込んでも無視されます。



第2章 開発環境の準備



開発環境を準備します。まず必要なものは「Windowsを搭載したパソコン」です。パソコンは低スペックでも問題ありません。他には、USB接続のゲームパッドがあったほうが便利です。

2-1 開発ツールを入手する

「Cコンパイラ」「テキストエディタ」「エミュレータ」を入手しましょう。

(1) Cコンパイラ

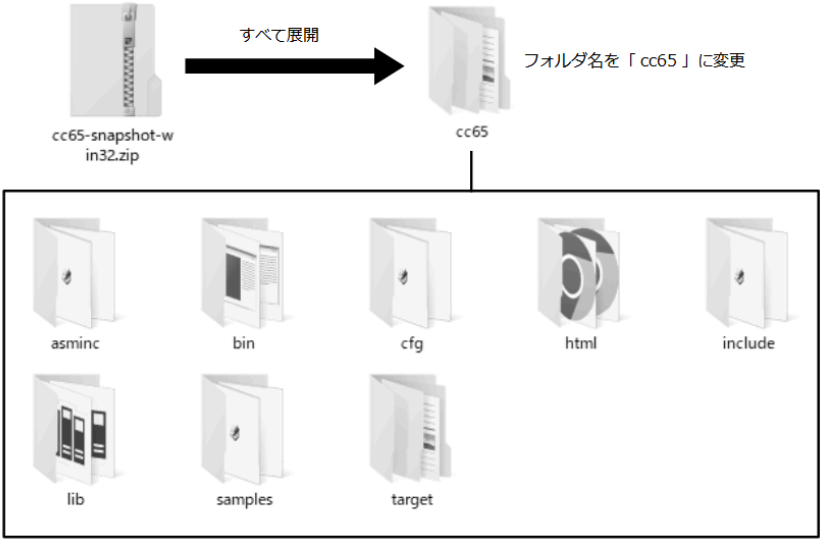
今回は「cc65」というフリーソフトのCコンパイラを使います。対応OSはWindowsです。Cコンパイラは「C言語」で書かれたプログラム(ソースコード)を実行可能なファイルに変換するソフトのことです。「cc65」はファミコン/NES、コモドール64、Apple II、Atari2600、PCエンジン、Lynxなど、6502系CPUを搭載したハードウェアに対応しています。

Webブラウザを使って次のURLを入力しましょう。

https://cc65.github.io/



Webページの下にある「Windows Snapshot」というリンクをクリックしましょう。クリックすると、「cc65-snapshot-win32.zip」というファイルがダウンロードされます。



続いて、エクスプローラーを使って、ダウンロードしたzipファイルを展

開します。ファイルを右クリックして「すべて展開」を選択すると、フォルダを新規作成して展開します。このフォルダの名前を「cc65」に変更します。最後に「cc65」フォルダをCドライブの一番上の階層（C:¥）に移動します。これでCコンパイラの準備は完了です。

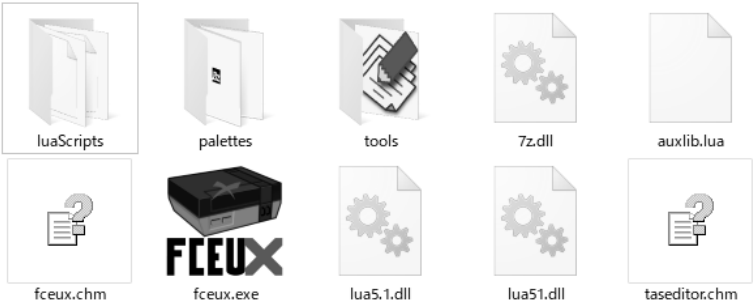
（2）テキストエディタ

テキストエディタは文字を読み書きするソフトのことです。Windowsに付属する「メモ帳」でも代用は可能ですが、改行が表示されないという欠点があります。筆者の場合はフリーソフトの「TeraPad」を使っています。

（3）エミュレータ

ファミコンのエミュレータが必要です。筆者は「FCEUX」というエミュレータを使っています。デバッグ機能が充実しているのが特徴です。「FCEUX」は次のページからダウンロードすることができます。フリーソフトです。

https://fceux.com/web/download.html



Webページにある「FCEUX～Binary」のリンクをクリックすると、zip

ファイルがダウンロードされます。zipファイルを右クリックして「すべて展開」を選択すると、フォルダを新規作成して展開されます。このフォルダを好きな場所(「ドキュメント」フォルダなど)に移しましょう。

以上で開発環境の準備が全て整いました。

《コラム》 ソフトの注意点

ソフトによっては「WindowsによってPCが保護されました」という警告が表示されます。この警告を解除するとソフトが使えますが、ソフトの中にコンピュータウイルスが入っていると、大変な被害が生じてしまいます。ウイルスチェックを行うなど、対策をしましょう。

《コラム》 ファイルの拡張子

基本中の基本ですが、ファイルは「拡張子が見える状態」にしておきましょう。Windowsのエクスプローラーのメニューから「表示」→「オプション」を選択してから、「登録されている拡張子は表示しない」 のチェックボックスをオフ(表示する)にしましょう。

《コラム》 6502

ファミコンのCPU「RP2A03E」はMOS Technology社の「MOS6502(以下、6502)」と互換性があります。ただし、機能を一部削って、APUの機能を追加しているので、完全な互換品ではありません。6502はレジスタの数が極端に少ないですが、命令の実行速度が比較的速いの特徴です。

2-2　HELLO WORLD

まずは練習として、「HELLO WORLD」という文字を表示するプログラムを作ります。ここではBGだけを使い、スプライトは使いません。エクスプローラーで作業用フォルダを新規作成して、名前を付けます（たとえば、「helloworld」）。そして、テキストエディタを使って、「main.c」「startup.asm」「build.cfg」「build.bat」という4つのファイルを作成しましょう。ファイルは作業用フォルダの中に保存します。

(1)「main.c」

「HELLO WORLD」のメインプログラムです。

```
// HELLO WORLD for FC (main.c)
// by Takuya Matsubara
#define PPUCTRL      *(char*)0x2000    // PPU制御レジスタ
#define PPUMASK      *(char*)0x2001    // PPUマスクレジスタ
#define PPUSCROLL    *(char*)0x2005    // PPUスクロールレジスタ
#define PPUADDR      *(char*)0x2006    // PPUアドレスレジスタ
#define PPUDATA      *(char*)0x2007    // PPUデータレジスタ

extern void waitvsync (void); // Vブランク待ち

// BG用 パターンテーブル
const unsigned char patternbg[] = {
0x38,0x44,0x82,0x82,0xfe,0x82,0x82,0x00,0x38,0x44,0x82,0x82,0xfe,0x8
2,0x82,0x00 /*chara65*/
,0xf0,0x88,0x88,0xfc,0x82,0x82,0xfc,0x00,0xf0,0x88,0x88,0xfc,0x82,0x
82,0xfc,0x00 /*chara66*/
,0x7c,0x82,0x80,0x80,0x80,0x82,0x7c,0x00,0x7c,0x82,0x80,0x80,0x80,0x
82,0x7c,0x00 /*chara67*/
,0xf8,0x84,0x82,0x82,0x82,0x84,0xf8,0x00,0xf8,0x84,0x82,0x82,0x82,0x
84,0xf8,0x00 /*chara68*/
,0xfe,0x80,0x80,0xfe,0x80,0x80,0xfe,0x00,0xfe,0x80,0x80,0xfe,0x80,0x
```

```
80,0xfe,0x00 /*chara69*/
,0xfe,0x80,0x80,0xfe,0x80,0x80,0x80,0x00,0xfe,0x80,0x80,0xfe,0x80,0x
80,0x80,0x00 /*chara70*/
,0x7c,0x82,0x80,0x8e,0x82,0x82,0x7c,0x00,0x7c,0x82,0x80,0x8e,0x82,0x
82,0x7c,0x00 /*chara71*/
,0x82,0x82,0x82,0xfe,0x82,0x82,0x82,0x00,0x82,0x82,0x82,0xfe,0x82,0x
82,0x82,0x00 /*chara72*/
,0x38,0x10,0x10,0x10,0x10,0x10,0x38,0x00,0x38,0x10,0x10,0x10,0x10,0x
10,0x38,0x00 /*chara73*/
,0x38,0x10,0x10,0x10,0x10,0x90,0x60,0x00,0x38,0x10,0x10,0x10,0x10,0x
90,0x60,0x00 /*chara74*/
,0x88,0x90,0xa0,0xc0,0xa0,0x90,0x88,0x00,0x88,0x90,0xa0,0xc0,0xa0,0x
90,0x88,0x00 /*chara75*/
,0x80,0x80,0x80,0x80,0x80,0x80,0xfe,0x00,0x80,0x80,0x80,0x80,0x80,0x
80,0xfe,0x00 /*chara76*/
,0x82,0xc6,0xc6,0xaa,0xaa,0x92,0x82,0x00,0x82,0xc6,0xc6,0xaa,0xaa,0x
92,0x82,0x00 /*chara77*/
,0x82,0xc2,0xa2,0x92,0x8a,0x86,0x82,0x00,0x82,0xc2,0xa2,0x92,0x8a,0x
86,0x82,0x00 /*chara78*/
,0x7c,0x82,0x82,0x82,0x82,0x82,0x7c,0x00,0x7c,0x82,0x82,0x82,0x82,0x
82,0x7c,0x00 /*chara79*/
,0xfc,0x82,0x82,0xfc,0x80,0x80,0x80,0x00,0xfc,0x82,0x82,0xfc,0x80,0x
80,0x80,0x00 /*chara80*/
,0x7c,0x82,0x82,0x82,0xba,0x84,0x7a,0x00,0x7c,0x82,0x82,0x82,0xba,0x
84,0x7a,0x00 /*chara81*/
,0xfc,0x82,0x82,0xfc,0x88,0x84,0x82,0x00,0xfc,0x82,0x82,0xfc,0x88,0x
84,0x82,0x00 /*chara82*/
,0x7e,0x80,0x80,0x7c,0x02,0x02,0xfc,0x00,0x7e,0x80,0x80,0x7c,0x02,0x
02,0xfc,0x00 /*chara83*/
,0xfe,0x10,0x10,0x10,0x10,0x10,0x10,0x00,0xfe,0x10,0x10,0x10,0x10,0x
10,0x10,0x00 /*chara84*/
,0x82,0x82,0x82,0x82,0x82,0x82,0x7c,0x00,0x82,0x82,0x82,0x82,0x82,0x
82,0x7c,0x00 /*chara85*/
,0x82,0x82,0x44,0x44,0x28,0x28,0x10,0x00,0x82,0x82,0x44,0x44,0x28,0x
28,0x10,0x00 /*chara86*/
,0x82,0x92,0xaa,0xaa,0xc6,0xc6,0x82,0x00,0x82,0x92,0xaa,0xaa,0xc6,0x
c6,0x82,0x00 /*chara87*/
,0x82,0x44,0x28,0x10,0x28,0x44,0x82,0x00,0x82,0x44,0x28,0x10,0x28,0x
44,0x82,0x00 /*chara88*/
,0x82,0x44,0x28,0x10,0x10,0x10,0x10,0x00,0x82,0x44,0x28,0x10,0x10,0x
10,0x10,0x00 /*chara89*/
,0xfe,0x04,0x08,0x10,0x20,0x40,0xfe,0x00,0xfe,0x04,0x08,0x10,0x20,0x
40,0xfe,0x00 /*chara90*/
};

// BG用カラーパレット
const char palette[] = { 0x0F ,0x00 ,0x10 ,0x30 };
```

```
// PPUアドレス設定(アドレス0x0000～0x3fff)
void ppu_address(unsigned short adr)
{
    PPUADDR = adr >> 8;    // アドレス上位8bit
    PPUADDR = adr & 0xff;  // アドレス下位8bit
}

// カラーパレット設定(格納先ポインタ,オフセット0～7,パレット数1～8)
void ppu_palette(char *ptable,char ofs,char size)
{
    size *= 4;     // 1パレットあたり4バイト
    ppu_address( 0x3f00+(ofs*4) );
    while(size--){
        PPUDATA = *ptable++;
    }
}

// パターンテーブル初期化
void ppu_pattern_init(void)
{
    int size = 512*16; // 16バイトx512パターン
    ppu_address(0);    // パターンテーブルのアドレス
    while(size--){
        PPUDATA = 0;
    }
}

// パターンテーブル設定(格納先ポインタ,オフセット0～511,枚数1～512)
void ppu_pattern(unsigned char *ptable,short ofs,short size)
{
    size *= 16;   // 1枚あたり16バイト
    ppu_address(ofs*16);   // パターンテーブルのアドレス
    while(size--){
        PPUDATA = *ptable++;
    }
}

// BGの初期化
void bg_cls(void)
{
    short i;
    ppu_address( 0x2000 );
    for(i=0; i<0x800; i++){
        PPUDATA = 0;    // ネームテーブルとアトリビュートの消去
    }
}

// BG文字列表示(X座標0～31,Y座標0～29,文字列ポインタ)
```

```
void bg_printstr(char x,char y,char *str)
{
    ppu_address( 0x2000+x+((unsigned short)y*32) );    //ネームテーブ
ルのアドレス
    while (*str != 0){
        PPUDATA = *str++;
    }
}

// BGスクロール設定(X座標0～255, Y座標0～239)
void bg_scroll(unsigned char x, unsigned char y)
{
    PPUSCROLL = x;   // X座標(0～255)
    PPUSCROLL = y;   // Y座標(0～239)
    PPUCTRL = 0x08;  // 画面Aを選択
}


// メイン関数
char NesMain()
{
    waitvsync();        // Vブランク待ち
    ppu_pattern_init(); // パターンテーブル初期化
    ppu_pattern((unsigned char *)patternbg,'A','Z'-'A'+1);  // BG/パ
ターンテーブル
    ppu_palette((char *)palette,0,1); // BGカラーパレット設定
    bg_cls();           // BGの初期化
    bg_printstr(10,14,"HELLO WORLD");
    waitvsync();        // Vブランク待ち
    PPUMASK = (1<<3)+6;  // 表示の有効化(bit3=BG)
    bg_scroll(0, 0);     // BGスクロール設定
    while(1){   // メインループ
        waitvsync();     // Vブランク待ち
    }
    return 0;
}
```

(2)「startup.asm」

アセンブラで記述したプログラムです。プログラムは3つの領域(セグメント)に分かれています。「HEADER」という領域はnesファイルのヘッダー情報です。マッパー0、CHR-RAM型、Vミラーリングを選択しています。「STARTUP」という領域はファミコンの起動プログラムです。ここで

は、スタックポインタを初期化して、「main.c」の中にある「NesMain」関数を呼び出しています。「VECINFO」という領域はリセットなどで呼び出すアドレスを記述しています。これをベクタテーブルと呼びます。

```
; startup.asm
.setcpu        "6502"
.autoimport    on
.IMPORTZP    sp
.global    _NesMain

.segment "HEADER"           ; iNESヘッダ(16 Bytes)
    .byte   $4E,$45,$53,$1A ; "NES"
    .byte   $02             ; PRG-ROMバンク数(16KB x 2)
    .byte   $00             ; CHR-ROMバンク数(8KB x 0)
    .byte   $01             ; Mapper 0(low)/V-Mirroring
    .byte   $00             ; Mapper 0(high)
    .byte   $00             ; PRG-RAMサイズ
    .byte   $00,$00,$00,$00, $00, $00, $00

.segment "STARTUP"     ; リセット割り込み
.proc    Reset
    sei
    ldx    #$ff        ; スタックポインタ初期化1
    txs
    lda    #$ff        ; スタックポインタ初期化2
    sta    sp
    lda    #$03
    sta    sp + 1
    jsr    _NesMain    ; メイン関数を呼び出す
.endproc

.segment "VECINFO"     ; ベクタテーブル
    .word    $0000     ; NMI割り込みアドレス
    .word    Reset     ; リセット割り込みアドレス
    .word    $0000     ; IRQ割り込みアドレス
```

(3)「build.cfg」

Cコンパイラの設定ファイルです。ROMやRAMのアドレスなど、ファミコンのハードウェアの情報を記述しています。

```
# build.cfg

MEMORY {
    HEADER:  start = $0000, size = $0010, file = %O, fill = yes;
    ROMST:   start = $8000, size = $7ffa, type = ro, file = %O, fill
= yes, define = yes;
    ROMINFO: start = $fffa, size = $0006, type = ro, file = %O, fill
= yes, define = yes;
    ROMCHR:  start = $0000, size = $2000, type = rw, define = yes;
    ZP:      start = $0000, size = $0020, type = rw, define = yes;
    RAM:     start = $0400, size = $0400, type = rw, define = yes;
    STACK:   start = $0200, size = $0200, type = rw, define = yes;
}

SEGMENTS {
    HEADER:  load = HEADER,  type = ro;
    STARTUP: load = ROMST,   type = ro,  define = yes;
    VECINFO: load = ROMINFO, type = ro,  define = yes;
    RODATA:  load = ROMST,   type = ro,  define = yes;
    DATA:    load = ROMST,   run = RAM,  type = rw,    define = yes;
    BSS:     load = RAM,     type = bss, define = yes;
    ZEROPAGE:load = ZP,      type = zp;
    CODE:    load = ROMST,   type = ro,  define = yes;
}

FEATURES {
    CONDES: segment = RODATA,
        type = constructor,
        label = __CONSTRUCTOR_TABLE__,
        count = __CONSTRUCTOR_COUNT__;
    CONDES: segment = RODATA,
        type = destructor,
        label = __DESTRUCTOR_TABLE__,
        count = __DESTRUCTOR_COUNT__;
}
```

(4)「build.bat」

ビルド用のバッチファイルです。長いコマンドは「^（キャレット）」で改行しています。C言語というと、環境変数やmakefileの設定が面倒だったりするのですが、どちらも使っていません。なお、「\（バックスラッシュ）」は日本語フォントだと「¥」と表示されます。

```
echo off
echo ROMイメージのビルドを行います(build.bat)
set SRC1=startup
set SRC2=main
set CFGFILE=build.cfg
set COMPILE=C:\cc65\bin\cl65
set LINK=C:\cc65\bin\ld65
set LIBFILE=C:\cc65\lib\nes.lib
set NESFILE=rom.nes
%COMPILE% -t none -o %SRC1%.o -c %SRC1%.asm
if %errorlevel% neq 0 goto abort
%COMPILE% -t none -o %SRC2%.o -c -O %SRC2%.c
if %errorlevel% neq 0 goto abort
%LINK% -o %NESFILE% --config %CFGFILE% ^
 --obj %SRC1%.o %SRC2%.o --lib %LIBFILE%
if %errorlevel% neq 0 goto abort
echo ビルドに成功しました
:abort
pause
```



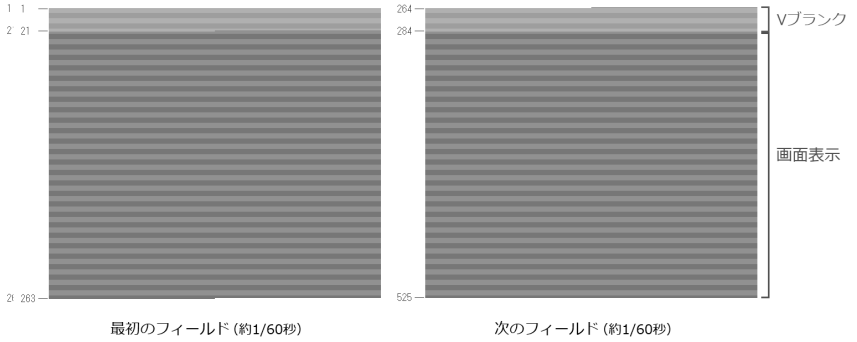
以上でファイルの準備は終了です。

「HELLO WORLD」の仕組み

このプログラムは大きく分けて「初期化」と「メインループ」の処理があります。初期化では「パターンテーブルの初期化」「パレットの初期化」「BG(ネームテーブル)の初期化」「BGに書き込み」「表示の有効化」を行っています。しっかり初期化をしないと、エミュレータでは問題なく動いても、実機で動かした時にゴミデータが表示されてしまいます。本当なら文字表示

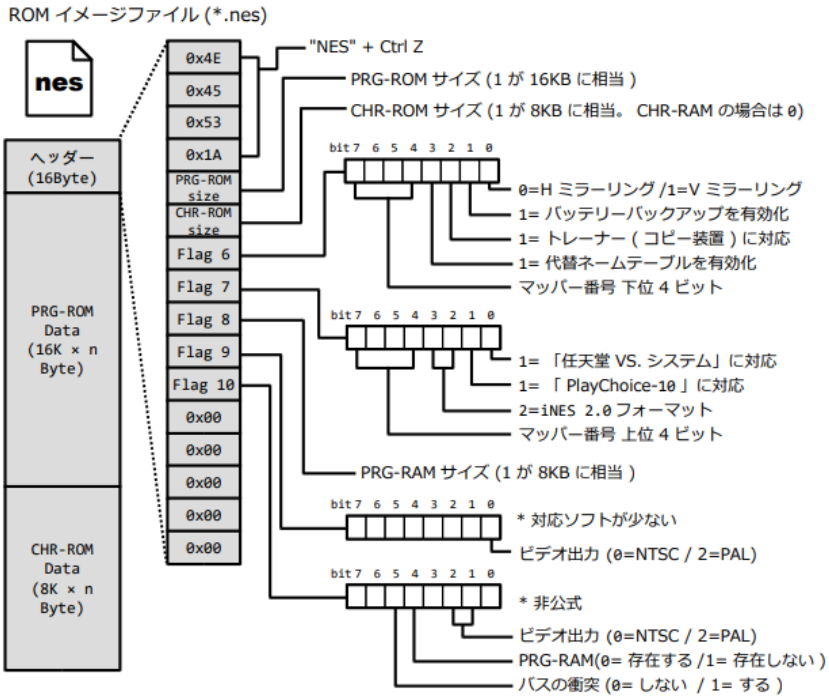
はメインループでやるべきですが、Vブランク中に描画が終わらない可能性があるので、初期化の中で行いました。メインループでは全くやることがないので、「Vブランクを待つ」処理だけを繰り返しています。これでメインループの実行周期が約1/60秒に調整されます。

《コラム》 Vブランクとは



Vブランクは「Vertical Blanking Interval」のことで、日本語では「垂直帰線期間」と言います。ファミコンは「NTSC」という規格に基づいて映像を出力しています。NTSCでは1秒間に約60（59.94）回のペースで走査線を最初に戻すための時間があります。それがVブランクです。画面を描画する単位をフィールドと呼びます。1フィールドが262.5ラインです。そのうちVブランクは約20ラインです。計算すると（1秒÷60）*（20÷262.5）=約1.269ミリ秒となります。Vブランクは驚くほど短い時間です。NTSCでは偶数・奇数の走査線を交互に描きますので、2フィールドで1つの画面が完成します。ファミコンのCPUがPPUを制御できるのは基本的にVブランクの時だけです。テレビが画面を描画している最中、CPUはPPUと通信できません。例外としてBGオフセット座標は描画中に変更可能です。その技術を応用して、画面を分割表示したり、ラスタースクロールを表現するソフトがあります。

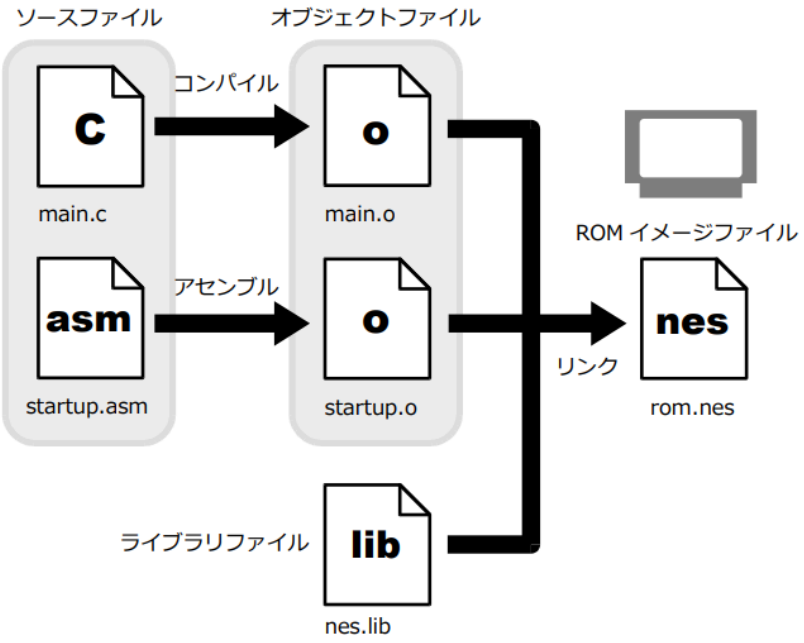
《コラム》nesファイル



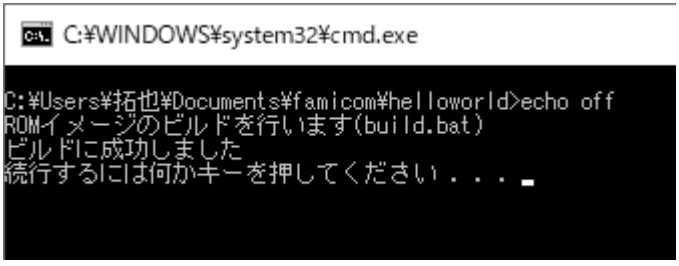
nesファイルはファミコンのエミュレータで普及しているファイル形式です。正しくはiNESフォーマットです。ファミコンはカセットの配線が統一されていないので、単なるROMイメージでは正しく動かすことができません。そこで、nesファイルではカセットの種類をヘッダー情報に書き込んでいます。

ビルドと実行

「ビルド」とは実行ファイルを作成する処理のことです。この場合、nesファイルを作る処理を行います。

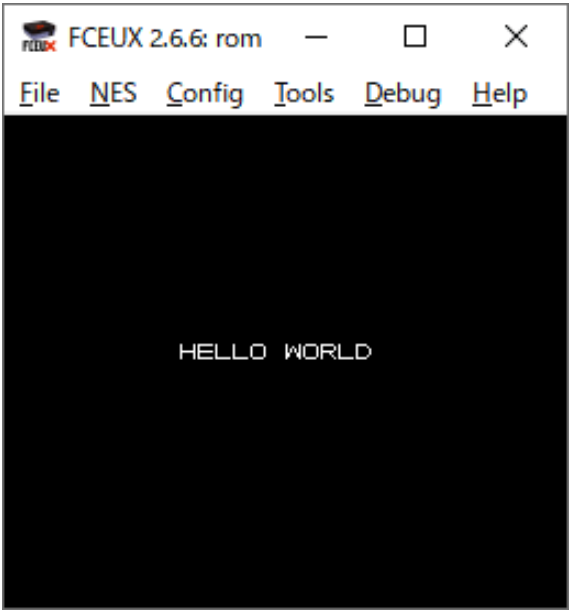


上の図はビルドの流れです。C言語のソースファイルはコンパイルされてオブジェクトファイルに変換され、アセンブラのソースファイルはアセンブルされてオブジェクトファイルに変換されます。そして、リンカを使って、オブジェクトファイルとライブラリファイルを1つのnesファイルに合体します。ライブラリファイルとは特定のプログラムをまとめて、使えるようにしたファイルです。ここでは「nes.lib」というCC65に付属するファミコン/NES用のライブラリを使います。



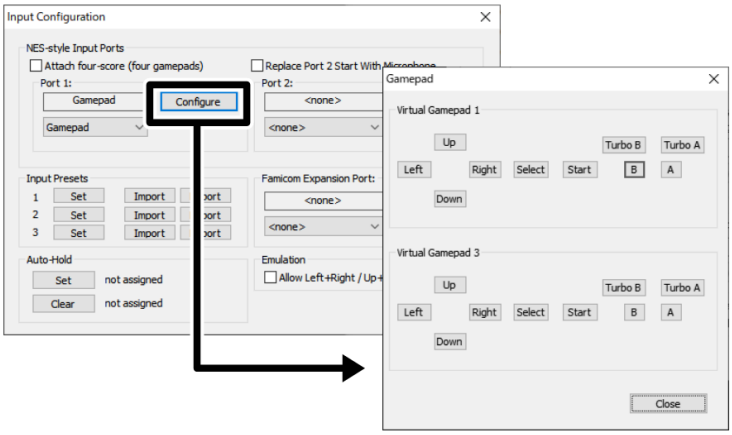
「build.bat」の実行結果です。コマンドプロンプトが立ち上がり、コンパイラ（cl65）とリンカ（ld65）を実行します。ビルドに成功すると「ビルドに成功しました」と表示されて、作業用フォルダの中にnesファイル（rom.nes）が作成されます。

エミュレータを使いこなす

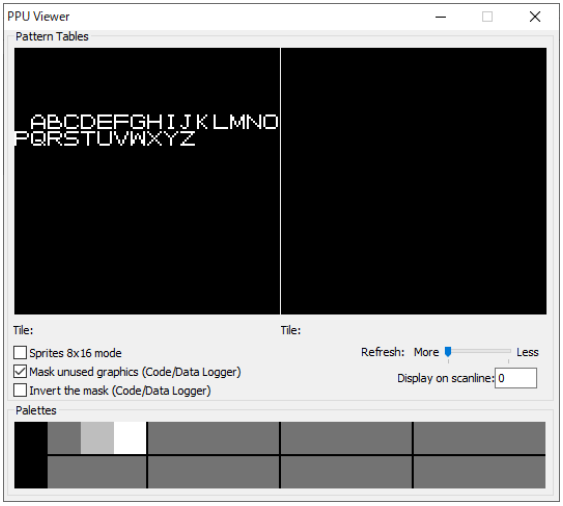


nesファイル（rom.nes）をエミュレータで実行します。ここではエミュレータ「FCEUX」の使い方を紹介します。エミュレータを立ち上げたら、メニューから「File」→「Open」を選択して、nesファイルを開きます。拡張子を関連付けしておけば、nesファイルをダブルクリックするだけでエミュレータを起動させることができます。nesファイルを実行すると、画面に「HELLO WORLD」と表示されます。無事にソフトが完成しました。「FCEUX」を初めて実行すると、画面サイズが小さくて見づらいです。サイ

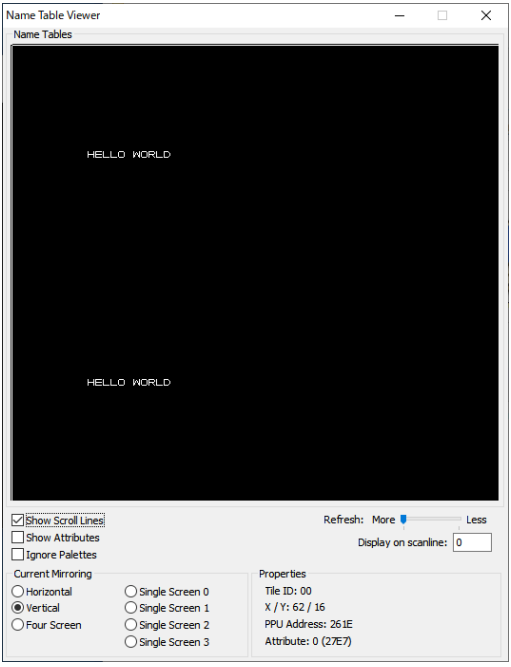
ズを大きくするにはウインドウの右下を外側にドラッグします。



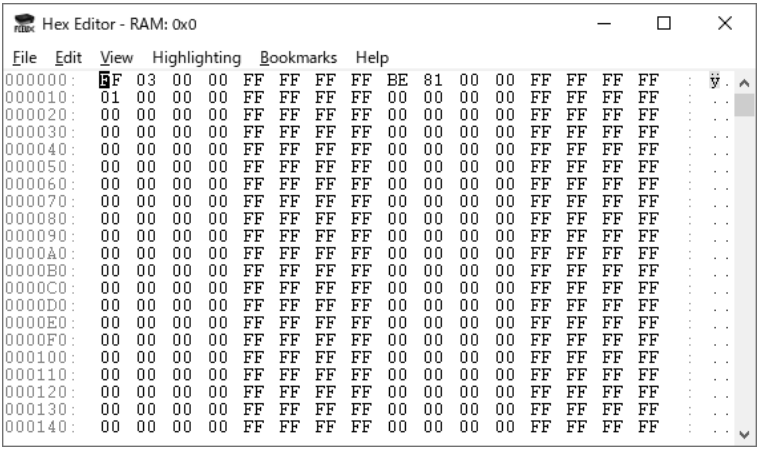
初めてエミュレータを実行した場合、メニューから「Config」→「Input」を選択して、ゲームパッドを登録しましょう。設定画面で「Configure」ボタンをクリックすると、ボタンを割り当てることができます。



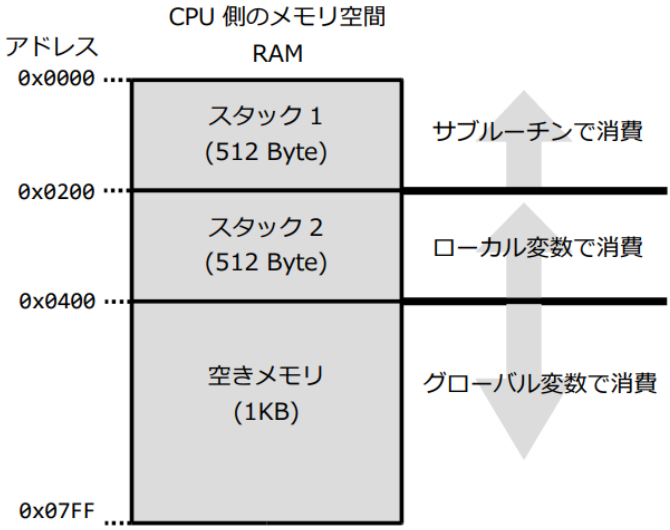
メニューから「Debug」→ 「PPU Viewer」を選択すると、パターンテーブルの中身を見ることができます。A～Zのタイル画像があります。



メニューから「Debug」→「Name　Table　Viewer」を選択すると、ネームテーブルの中身を見ることができます。2×2=4画面をまとめて表示しています。「HELLO　WORLD」の文字が上下で2つありますが、下の文字は「ミラー」です。



メニューから「Debug」→「Hex　Editor」を選択すると、RAMやROMの中身を見ることができます。RAMの変化をリアルタイムで確認できるので便利です。RAMの中身がほぼ「0x00」や「0xFF」で埋まっていますが、これはメモリにまだ余裕があるということです。



ファミコンのRAMは0x0000～0x07FF番地にあります。メモリの一時的な退避場所を「スタック」と呼びます。CC65ではスタックは2つ使っています。0x0000～0x01FF番地が通常のスタックです。0x0200～0x03FF番地はC言語のローカル変数などで使うスタックです。スタックは大きなアドレスから自動的に埋まります。0x0400～0x07FF番地がC言語のグローバル変数などで使う空きメモリです。空きメモリは小さいアドレスから埋まります。

他にも「FCEUX」には、CPUを途中で止めたり、命令を1ステップずつ実行できる「デバッガ」という機能もありますが、アセンブラの知識が必要なので上級者向けです。

2-3 タイル画像を描く



ジャンピングアクションゲームやシューティングゲームで使えるタイル画像をお絵描きソフトで描きます。推奨する色は次の4色です。

・白色(255,255,255)

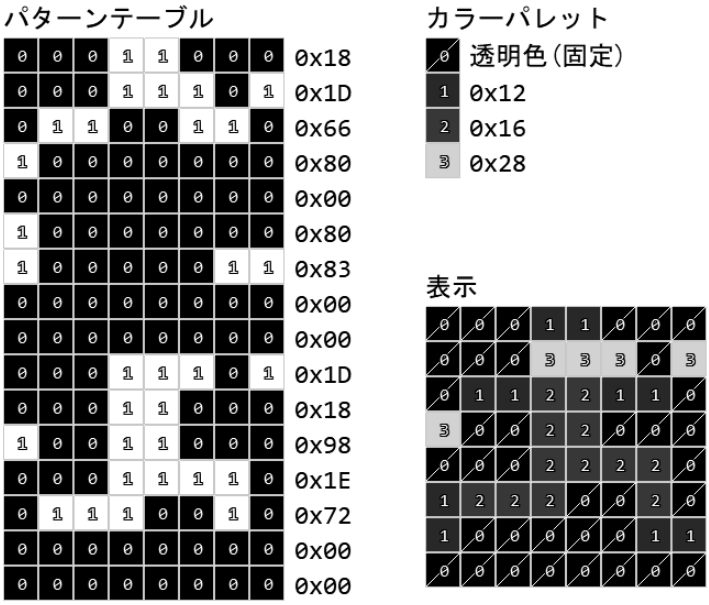
・薄い灰色(192,192,192)

・濃い灰色(128,128,128)

・黒色(0,0,0)。黒色は透明色と解釈します。

画像のサイズは128×256ピクセルです。画像は上半分をBG用、下半分をスプライト用に割り振ってみました。キャラクターのサイズは16×16ピクセル。アニメーションは2コマです。画像のファイル名は「pattern.bmp」として、16色ビットマップ形式で保存します。

画像をデータに変換



タイル画像をデータに変換します。上の図は画像をデータに変換する例です。画像は8×8ピクセルごとに2枚の「プレーン」に置き換えます。プレーン1枚目がインデックスカラーのビット0、2枚目がインデックスカラーのビット1を表していて、重ねると2ビット＝4色となります。プレーンは2進数で描きます。これを8ビットずつ、行単位でまとめると、16バイトのデータになります。

この方法を踏まえて、BMP画像をC言語に変換するプログラムをPythonで作ります。Pythonの開発環境は初心者向けの「Thonny」がおすすめです。以下のWebページから入手できます。

https://thonny.org/

BMP→C言語の変換プログラムは次の通りです。

```
# make pattern (16色BMPファイルをソースファイルに変換)
cnv = [ 0, 128, 192, 255 ]      # 4諧調カラー変換テーブル
inputname = 'pattern.bmp'       # 入力ファイル名
outputname = 'pattern.h'        # 出力ファイル名
print("load:"+inputname)
infile = open(inputname, 'rb')  # ファイルの読み込み
indata = infile.read()
infile.close()
header = int.from_bytes(indata[10:14], 'little')  # ヘッダ長
bmpw = int.from_bytes(indata[18:22], 'little')    # 画像の幅
bmph = int.from_bytes(indata[22:26], 'little')    # 画像の高さ
depth = int.from_bytes(indata[28:30], 'little')   # カラーbit
if depth != 4:
    print('ERROR:カラーが16色(4bit)ではありません')

pallete = []      # 16色カラーパレットを読み込み
for i in range(16):
    idx = 54+(4*i)
    color = int.from_bytes(indata[idx:idx+4], 'little')
    pallete.append(color)

cellsize = 8  # セルの大きさ[ピクセル]
cellw = int(bmpw / cellsize)  # 画像の幅[セル]
cellh = int(bmph / cellsize) # 画像の高さ[セル]
comma = ' '
fbuf = '// '+ outputname +'\n'
fbuf += 'const unsigned char pattern[] = {\n'
for i in range(cellw * cellh):
    ofsx = (i % cellw)*cellsize
    ofsy = int(i / cellw)*cellsize
    for plane in range(2):
        for y  in range(cellsize):
            tmp = 0
            for x  in range(cellsize):
                ry = ((bmph-1)-(y+ofsy))
                idx = int(((x+ofsx)+(ry*bmpw))/2) + header
                if (x & 1)==0 :
                    coloridx = (indata[idx] >> 4)& 0xf
                else:
                    coloridx = indata[idx] & 0xf

                color = pallete[coloridx]     # 指定座標の色
                r = (color >> 16) & 0xff
                g = (color >> 8) & 0xff
                b = color & 0xff
                v = int((r + g + b)/3)        # 256諧調に変換
                for code in range(4):
                    if v <= cnv[code]: break  # 4諧調に変換
```

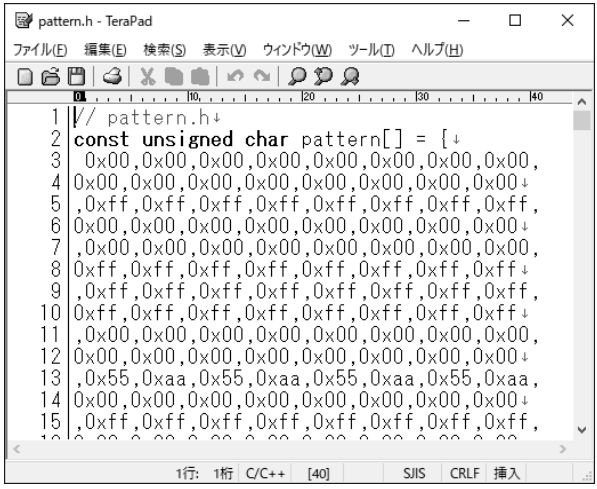
```
                if ((1 << plane) & code) != 0:
                    tmp += (0x80 >> x)        # バイナリに変換

            fbuf += comma + '0x' + '{:02x}'.format(tmp)
            comma = ','

    fbuf += '\n'

fbuf += '};\n'
outfile = open(outputname, 'w')  # ファイルの出力
outfile.write(fbuf)
outfile.close()
print("save:"+outputname)
```

「Thonny」でプログラムを入力して、「make_pattern.py」のファイル名で保存します。ファイルを開いた状態でキーボードの「F5」キーを押すと実行します。実行すると、画像ファイルが変換されて、「pattern.h」というファイルが保存されます。



「pattern.h」をテキストエディタで開いて確認します。これが変換した結果です。512枚のタイル画像を8KBの配列に格納しています。

《コラム》ドット絵用のお絵描きツール

筆者が自作したドット絵専用のお絵描きツール「spedit」です。画像からソースファイルやバイナリファイルを作成することができます。GitHubでソースを公開しています。

https://github.com/nicotakuya/hsp_spedit

2-4 デモプログラム

ゲームを手軽に作れるようにするため、オリジナルのライブラリを作ります。これを「FCライブラリ」と名付けました。FCライブラリを使えば、画面を表示したり、コントローラーを読み取ったり、音を鳴らすような処理を関数を呼ぶだけで済ませることができます。ライブラリ単体だとうまく使い方が伝わらないので、「デモプログラム」を作りました。作業用のフォルダを作って、その中に「main.c」「fcsub.h」「fcsub.c」「startup.asm」「build.cfg」「build.bat」という6つのファイルを作成しましょう。

(1)「main.c」

メインプログラムです。BGとスプライトを表示するデモです。

```
// FC Library Demo for FC (main.c)
// by Takuya Matsubara
#include "fcsub.h"     // FCライブラリ
#include "pattern.h"   // パターンデータ

extern void waitvsync (void); // Vブランク待ち

char x1[32],y1[32];   // 移動量

// メイン関数
char NesMain(void)
```

```
{
    char num;
    unsigned char x,y,tile;
    short bgx;

    fc_init();      // 全機能の初期化
    // パターンテーブル設定
    ppu_pattern((unsigned char *)pattern,0,256);
    ppu_pattern8x16((unsigned char *)pattern+4096,256,256);

    // BGの描画
    for(y=0; y<30; y+=2){
        for(x=0; x<32; x+=2){
            if((x>4)&&(x<26)&&(y>4)&&(y<24)) continue;
            tile = (random() % 3)*2 + 0xe0;
            bg_print2x2(x,y,tile);
        }
    }
    bg_printstr(8,15,"FC LIBRARY DEMO");

    // スプライト追加
    for(num=0; num<32; num++){
        // 座標、移動方向
        x = (num % 4)*32+16;
        y = (num / 4)*24+16;
        x1[num] = (char)(random() % 5)-2;
        y1[num] = (char)(random() % 5)-2;
        // SP追加(0x1000番地以降のタイル画像は+1)
        sp_append(x,y,(num*8)+1,PAL_0);
    }
    waitvsync();      // Vブランク待ち
    ppu_enable(1);    // 表示を有効化

    num = 0;  // スプライト番号
    while (1){
        x = sp_getx(num);
        y = sp_gety(num);
        x += x1[num];
        y += y1[num];
        if((x <= 8)||(x >= 240)){
            x1[num] = -x1[num];
        }
        if((y <= 8)||(y >= 216)){
            y1[num] = -y1[num];
        }
        sp_ofs(num,x,y);  // スプライト移動
        sp_anime(num);    // アニメ表示
        num = (num+1) % 32;  // スプライト番号を切り替え
```

```
        bgx = (bgx+1) & 511; // BGオフセット座標
        waitvsync();         // Vブランク待ち
        sp_dmastart();       // OMA DMA転送開始
        bg_scroll(bgx,0);    // BGスクロール
    }
    return 0;
}
```

(2)「fcsub.h」

FCライブラリのヘッダファイルです。これを読むと、ライブラリの機能がわかります。

```
// FC LIBRARY (fcsub.h)
// by Takuya Matsubara
#define PPUCTRL      *(char*)0x2000    // PPU制御レジスタ
#define PPUMASK      *(char*)0x2001    // PPUマスクレジスタ
#define PPUSTATUS    *(char*)0x2002    // PPUステータスレジスタ
#define OAMADDR      *(char*)0x2003    // OAMアドレスポート
#define OAMDATA      *(char*)0x2004    // OAMデータポート
#define PPUSCROLL    *(char*)0x2005    // PPUスクロールレジスタ
#define PPUADDR      *(char*)0x2006    // PPUアドレスレジスタ
#define PPUDATA      *(char*)0x2007    // PPUデータレジスタ
#define JOYPAD1      *(char*)0x4016    // コントローラー1
#define JOYPAD2      *(char*)0x4017    // コントローラー2
#define PULSE1CTR1   *(char*)0x4000    // 矩形波1制御レジスタ1
#define PULSE1CTR2   *(char*)0x4001    // 矩形波1制御レジスタ2
#define PULSE1FREQ1  *(char*)0x4002    // 矩形波1周波数1
#define PULSE1FREQ2  *(char*)0x4003    // 矩形波1周波数2
#define OAMDMA       *(char*)0x4014    // OAM DMAレジスタ
#define SNDCHANNEL   *(char*)0x4015    // サウンドチャンネル制御
#define SPWADDR      0x700             // スプライトワークの格納アドレス

#define C_BLACK      0x0f  // カラーコード 黒色
#define C_RED        0x16  // カラーコード 赤色
#define C_RED0       0x06  // カラーコード 暗い赤色
#define C_RED1       0x16  // カラーコード 赤色
#define C_RED2       0x26  // カラーコード 明るい赤色
#define C_BLUE       0x12  // カラーコード 青色
#define C_BLUE0      0x02  // カラーコード 暗い青色
#define C_BLUE1      0x12  // カラーコード 青色
#define C_BLUE2      0x22  // カラーコード 明るい青色
#define C_GREEN      0x1A  // カラーコード 緑色
```

```
#define C_GREEN0     0x0A  // カラーコード 暗い緑色
#define C_GREEN1     0x1A  // カラーコード 緑色
#define C_GREEN2     0x2A  // カラーコード 明るい緑色
#define C_YELLOW     0x28  // カラーコード 黄色
#define C_YELLOW0    0x08  // カラーコード 暗い黄色
#define C_YELLOW1    0x18  // カラーコード 黄色
#define C_YELLOW2    0x28  // カラーコード 明るい黄色
#define C_SKYBLUE    0x22  // カラーコード 空色
#define C_LIMEGREEN  0x29  // カラーコード 黄緑色
#define C_TEAL       0x1C  // カラーコード ティール
#define C_TEAL0      0x0C  // カラーコード 暗いティール
#define C_TEAL1      0x1C  // カラーコード ティール
#define C_TEAL2      0x2C  // カラーコード 明るいティール
#define C_GRAY       0x00  // カラーコード 灰色
#define C_LIGHTGRAY  0x10  // カラーコード 明るい灰色
#define C_GRAY0      0x00  // カラーコード 暗い灰色
#define C_GRAY1      0x10  // カラーコード 灰色
#define C_GRAY2      0x20  // カラーコード 明るい灰色
#define C_PURPLE     0x14  // カラーコード 紫色
#define C_PURPLE0    0x04  // カラーコード 暗い紫色
#define C_PURPLE1    0x14  // カラーコード 紫色
#define C_PURPLE2    0x24  // カラーコード 明るい紫色
#define C_MAGENTA    0x15  // カラーコード マゼンタ
#define C_MAGENTA0   0x05  // カラーコード 暗いマゼンタ
#define C_MAGENTA1   0x15  // カラーコード マゼンタ
#define C_MAGENTA2   0x25  // カラーコード 明るいマゼンタ
#define C_WHITE      0x30  // カラーコード 白色

#define BUTTON_A      0x80   // ボタン A
#define BUTTON_B      0x40   // ボタン B
#define BUTTON_SELECT 0x20   // ボタン SELECT
#define BUTTON_START  0x10   // ボタン START
#define BUTTON_UP     0x08   // ボタン 上
#define BUTTON_DOWN   0x04   // ボタン 下
#define BUTTON_LEFT   0x02   // ボタン 左
#define BUTTON_RIGHT  0x01   // ボタン 右

#define OUTSIDE      240     // 画面外の座標

#define PCTR_SP8X16 (1<<5)   // SP 8x16モード
#define PCTR_BGP1 (1<<4) // BGパターンテーブルを0x1000番地に
#define PCTR_SPP1 (1<<3) // SPパターンテーブルを0x1000番地に

// PPUMASK用
#define PMSK_SHOWSP  (1<<4)  // SPの表示
#define PMSK_SHOWBG  (1<<3)  // BGの表示
#define PMSK_SHOWLSP (1<<2)  // 左8pixelのSP表示
#define PMSK_SHOWLBG (1<<1)  // 左8pixelのBG表示
```

```
#define PMSK_SHOWALL 0x1E    // SP&BGを表示

// スプライト用アトリビュート
#define ATR_BEHIND (1<<5)  // SPをBGの奥に表示
#define ATR_FLIPH  (1<<6)  // SPを左右反転
#define ATR_FLIPV  (1<<7)  // SPを上下反転

// パレット番号
#define PAL_0  0   // パレット番号0
#define PAL_1  1   // パレット番号1
#define PAL_2  2   // パレット番号2
#define PAL_3  3   // パレット番号3

// 全機能の初期化
void fc_init(void);
// PPUアドレス設定(アドレス0x0000～0x3fff)
void ppu_address(unsigned short adr);
// BGの初期化
void bg_cls(void);
// スプライトの初期化
void sp_cls(void);
// 疑似乱数の取得
unsigned int random(void);
// コントローラー1入力
unsigned char controller1(void);
// サウンド初期化
void sound_init(void);
// 矩形波1再生(周波数)
void pulse1(unsigned short hz);
// カラーパレット設定(格納先ポインタ,オフセット0～7,パレット数1～8)
void ppu_palette(char *ptable,char ofs,char size);
// パターンテーブル設定(格納先ポインタ,オフセット0～511,枚数1～512)
void ppu_pattern(unsigned char *ptable,short ofs,short size);
// 8x16パターンテーブル設定(格納先ポインタ,オフセット0～511,枚数1～512)
void ppu_pattern8x16(unsigned char *ptable,short ofs,short size);
// 画面表示の有効化(1=表示ON/0=表示OFF)
void ppu_enable(char on);
// Vブランクを待つ
void ppu_vsync(void);
// BGスクロール設定(X座標オフセット0～511,Y座標オフセット0～239)
void bg_scroll(unsigned short x, unsigned char y);
// BGキャラクタ2x2セル表示(X座標0～31,Y座標0～29,タイル番号0～255)
void bg_print2x2(char x,char y,unsigned char c);
// BGキャラクタ表示(X座標0～31,Y座標0～29,タイル番号0～255)
void bg_printch(char x,char y,unsigned char c);
// BG文字列表示(X座標0～31,Y座標0～29,文字列ポインタ)
void bg_printstr(char x,char y,char *str);
// BGアトリビュート設定(X座標0～31,Y座標0～29,アトリビュート0～255)
```

```
void bg_attrib(char x,char y,unsigned char atr);
// 10進数5ケタ変換(格納先ポインタ,数値0～65535)
void num_to_str(char *str,unsigned short num);
// 16進数2ケタ変換(格納先ポインタ,数値0～255)
void num_to_hex(char *str,unsigned char num);
// スプライトワークのDMA転送を開始する
void sp_dmastart(void);
// スプライト 追加(X座標,Y座標,タイル0～255,アトリビュート0～255)
char sp_append(char x,char y,char tile,char attr);
// スプライト 消去(スプライト番号0～31)
void sp_remove(char i);
// スプライト 表示変更(スプライト番号0～31,タイル0～255,アトリビュート0～
255)
void sp_chr(char num,unsigned char tile,unsigned  char attr);
// スプライト アニメ処理(スプライト番号0～31)
void sp_anime(char num);
// スプライト 移動(スプライト番号0～31,X座標0～255,Y座標0～255)
void sp_ofs(char num,unsigned char x,unsigned  char y);
// スプライトX座標取得(スプライト番号0～63)
unsigned char sp_getx(char num);
// スプライトY座標取得(スプライト番号0～63)
unsigned char sp_gety(char num);
// スプライトのタイル取得(スプライト番号0～63)
unsigned char sp_gettile(char num);
// スプライト衝突判定(スプライト番号,X座標,Y座標,しきい値)
char sp_hitcheck(char num,unsigned char x,unsigned char y,unsigned
char diff);
```

(3)「fcsub.c」

FCライブラリのソースです。長いです。

```
// FC LIBRARY (fcsub.c)
// by Takuya Matsubara
#include "fcsub.h"

unsigned int rndwork; // 疑似乱数

// 全機能の初期化
void fc_init(void)
{
    ppu_vsync();      // Vブランク待ち
    ppu_enable(0);    // 表示を無効化
    rndwork = 0x005a; // 疑似乱数のシード値を設定
    sound_init();     // サウンドの初期化
```

```
    bg_cls();          // BGの初期化
    sp_cls();          // スプライトの初期化
    bg_scroll(0, 0);   // BGスクロール設定
}

// PPUアドレス設定(アドレス0x0000～0x3fff)
void ppu_address(unsigned short adr)
{
    PPUADDR = adr >> 8;    // アドレス上位8bit
    PPUADDR = adr & 0xff;  // アドレス下位8bit
}

// BGの初期化
void bg_cls(void)
{
    short i;
    ppu_address( 0x2000 );
    for(i=0; i<0x800; i++){
        PPUDATA = 0;    // ネームテーブルとアトリビュートの消去
    }
}

// スプライトの初期化
void sp_cls(void)
{
    char i;
    for(i=0; i<32; i++){
        sp_remove(i);    // スプライトを画面外に
    }
}

// 疑似乱数の取得
unsigned int random(void)
{
    rndwork ^= (rndwork << 3);
    rndwork ^= (rndwork >> 5);
    rndwork ^= (rndwork << 4);
    return(rndwork);   // 戻り値=乱数0～0xffff
}

// コントローラー1+外部コントローラ入力
unsigned char controller1(void)
{
    unsigned char ret=0;
    char i;
    JOYPAD1 = 0x01;    // ボタン入力を開始
    JOYPAD1 = 0x00;    // ボタン入力をラッチ
    for(i=0; i<8; i++){
```

```
        ret <<= 1;
        if( JOYPAD1 & 3) ret |= 1;  // ボタンが押された
    }
    return(ret);   // 戻り値=ボタンの状態(8bit)
}

// サウンド初期化
void sound_init(void)
{
    #define DUTY 2      // 波形のデューティ比設定(2=50%)
    #define VOLUME 8    // 音のボリューム(0～15)
    #define SHIFT 2     // 周波数変化量(0～7)
    #define DIVID 4     // 周波数変化周期(0～7)
    #define DIR 1       // 周波数変化方法(1=高音に変化)
    PULSE1CTR1 = (DUTY<<6)+(1<<5)+(1<<4)+VOLUME;    // 矩形波1制御レ
ジスタ1
    PULSE1CTR2 = (1<<7)+(DIVID<<4)+(DIR<<3)+SHIFT;  // 矩形波1制御レ
ジスタ2

    SNDCHANNEL = (1<<0);      // 矩形波1のみを使用する
}

// 矩形波1再生(周波数)
void pulse1(unsigned short hz)
{
    unsigned short tmp;
    tmp = (unsigned short)((1790000/16)/hz)-1;
    PULSE1FREQ1 = tmp & 0xff; // 矩形波1周波数1(bit7～0)
    PULSE1FREQ2 = tmp >> 8;   // 矩形波1周波数2(bit10～8)
}

// カラーパレット設定(格納先ポインタ,オフセット0～7,パレット数1～8)
void ppu_palette(char *ptable,char ofs,char size)
{
    size *= 4;     // 1パレットあたり4バイト
    ppu_address( 0x3f00+(ofs*4) );    // カラーパレットのアドレス
    while(size--){
        PPUDATA = *ptable++;
    }
}

// パターンテーブル設定(格納先ポインタ,オフセット0～511,枚数1～512)
void ppu_pattern(unsigned char *ptable,short ofs,short size)
{
    size *= 16;   // 1枚あたり16バイト
    ppu_address(ofs*16);   // パターンテーブルのアドレス
    while(size--){
        PPUDATA = *ptable++;
```

```
    }
}

// 8x16モード用パターンテーブル設定(ポインタ,オフセット0～511,枚数1～
512)
void ppu_pattern8x16(unsigned char *ptable,short ofs,short size)
{
    unsigned char *src;
    short i,j;
    ppu_address(ofs*16);   // パターンテーブルのアドレス

    for(i=0;i<size;i++){
        j = ((i/2) % 16)+((i/32)*32);
        j += ((i & 1)*16);
        src = ptable+(j*16);
        for(j=0; j<16; j++){
            PPUDATA = *src++;
        }
    }
}

// 画面表示の有効化(1=表示ON/0=表示OFF)
void ppu_enable(char on)
{
    PPUMASK = on*PMSK_SHOWALL;    // PPUマスク
}

// Vブランクを待つ
void ppu_vsync(void)
{
    while((PPUSTATUS & (1<<7))==0);    // bit7が0の場合、画面描画中
}

// BGスクロール設定(X座標オフセット0～511,Y座標オフセット0～239)
void bg_scroll(unsigned short x, unsigned char y)
{
    PPUSCROLL = (x & 0xff);   // X座標(0～255)
    PPUSCROLL = y;            // Y座標
    PPUCTRL = PCTR_SP8X16 + ((x >> 8)& 1);
    // SP8x16モード + X座標256以上の場合は画面B
}

// BGキャラクタ2x2セル表示(X座標0～31,Y座標0～29,タイル番号0～255)
void bg_print2x2(char x,char y,unsigned char c)
{
    unsigned short adr;
    //ネームテーブルのアドレス
    adr = 0x2000+x+((unsigned short)y*32);
```

```
    ppu_address( adr );
    PPUDATA = c;
    PPUDATA = c+1;
    ppu_address( adr+32 );
    PPUDATA = c+16;
    PPUDATA = c+17;
}

// BGキャラクタ表示(X座標0～31,Y座標0～29,タイル番号0～255)
void bg_printch(char x,char y,unsigned char c)
{
    // 裏ワザとして、Y座標が32～61の場合は画面Bに書き込みができる
    ppu_address( 0x2000+x+((unsigned short)y*32) );    //ネームテーブ
ルのアドレス
    PPUDATA = c;
}

// BG文字列表示(X座標0～31,Y座標0～29,文字列ポインタ)
void bg_printstr(char x,char y,char *str)
{
    ppu_address( 0x2000+x+((unsigned short)y*32) );    //ネームテーブ
ルのアドレス
    while (*str != 0){
        PPUDATA = *str++;
    }
}

// BGアトリビュート設定(X座標0～31,Y座標0～29,アトリビュート0～255)
void bg_attrib(char x,char y,unsigned char atr)
{
    ppu_address( 0x23C0+(x/4)+((y/4)*8) );    // BGアトリビュートのア
ドレス
    PPUDATA = atr;
}

// 10進数5ケタ変換(格納先ポインタ,数値0～65535)
void num_to_str(char *str,unsigned short num)
{
    char i;
    for(i=4;;i--){
        str[i] = (num % 10)+'0';
        num /= 10;
        if(i==0)break;
    }
    str[5] = 0; // NULL
}

// 16進数2ケタ変換(格納先ポインタ,数値0～255)
```

```
void num_to_hex(char *str,unsigned char num)
{
    unsigned char tmp;
    tmp = '0'+(num >> 4);             // 0〜9に変換
    if(tmp > '9') tmp += 'A'-('9'+1); // A〜Fに変換
    str[0] = tmp;  // 上位4bit
    tmp = '0'+(num & 0x0f);           // 0〜9に変換
    if(tmp > '9')tmp += 'A'-('9'+1);  // A〜Fに変換
    str[1] = tmp;  // 下位4bit
    str[2] = 0; // NULL
}

// スプライトワークのDMA転送を開始する
void sp_dmastart(void)
{
    OAMADDR = 0;             // 転送先OAMアドレス
    OAMDMA = SPWADDR >> 8;  // 転送元アドレス上位8bit
}

// *スプライトは2枚1組、16×16ピクセルで扱います。
// スプライト 追加(X座標,Y座標,タイル0〜255,アトリビュート0〜255)
char sp_append(char x,char y,char tile,char attr)
{
    char i;
    for(i=0; i<32; i++){
        if(sp_gety(i)==OUTSIDE){
            sp_ofs(i, x,y);
            sp_chr(i, tile,attr);
            break;
        }
    }
    return(i);  // スプライト番号
}

// スプライト 消去(スプライト番号0〜31)
void sp_remove(char i)
{
    sp_ofs(i,OUTSIDE,OUTSIDE);    // スプライトを画面外に
}

// スプライト 表示変更(スプライト番号0〜31,タイル0〜255,アトリビュート0〜
255)
void sp_chr(char num,unsigned char tile,unsigned char attr)
{
    unsigned char *ptr;
    ptr = (unsigned char *)SPWADDR+(num*4); // スプライトワーク書き込
みアドレス
    if(attr & ATR_FLIPH){
```

```
        ptr[1+(128*0)] = tile+2;  // スプライトnum+0 タイル番号
        ptr[1+(128*1)] = tile;    // スプライトnum+1 タイル番号
    }else{
        ptr[1+(128*0)] = tile;    // スプライトnum+0 タイル番号
        ptr[1+(128*1)] = tile+2;  // スプライトnum+1 タイル番号
    }
    ptr[2+(128*0)] = attr;    // スプライトnum+0 アトリビュート
    ptr[2+(128*1)] = attr;    // スプライトnum+1 アトリビュート
}

// スプライト アニメ処理(スプライト番号0～31)
void sp_anime(char num)
{
    unsigned char *ptr;
    ptr = (unsigned char *)SPWADDR+(num*4); // スプライトワークアドレ
ス
    ptr[1+(128*0)] ^= 4;   // スプライトnum+0 タイル番号
    ptr[1+(128*1)] ^= 4;   // スプライトnum+1 タイル番号
}

// スプライト 移動(スプライト番号0～31,X座標0～255,Y座標0～255)
void sp_ofs(char num,unsigned char x,unsigned  char y)
{
    unsigned char *ptr;
    ptr = (unsigned char *)SPWADDR+(num*4); // スプライトワークアドレ
ス
    ptr[0+(128*0)] = y;    // スプライトnum+0 Y座標
    ptr[3+(128*0)] = x;    // スプライトnum+0 X座標
    ptr[0+(128*1)] = y;    // スプライトnum+1 Y座標
    ptr[3+(128*1)] = x+8;  // スプライトnum+1 X座標
}

// スプライトX座標取得(スプライト番号0～63)
unsigned char sp_getx(char num)
{
    return(*((unsigned char *)SPWADDR+3+(num*4)));
}

// スプライトY座標取得(スプライト番号0～63)
unsigned char sp_gety(char num)
{
    return(*((unsigned char *)SPWADDR+0+(num*4)));
}

// スプライトのタイル取得(スプライト番号0～63)
unsigned char sp_gettile(char num)
{
    return(*((unsigned char *)SPWADDR+1+(num*4))); // 戻り値=タイル番
```

```
号
}

// スプライト衝突判定(スプライト番号,X座標,Y座標,しきい値)
char sp_hitcheck(char num,unsigned char x,unsigned char y,unsigned
char diff)
{
    unsigned char *ptr;
    ptr = (unsigned char *)SPWADDR+(num*4); // スプライトワークアドレ
ス
    x += diff;
    x -= ptr[3];     // X座標
    if(x > diff*2)return(0);  // 衝突なし
    y += diff;
    y -= ptr[0];     // Y座標
    if(y > diff*2)return(0);  // 衝突なし
    return(1);  // 戻り値=1の場合、衝突あり
}
```

(4)「startup.asm」

「(2-3)HELLO WORLD」の「startup.asm」と同じです。

(5)「build.cfg」

「(2-3)HELLO WORLD」の「build.cfg」と同じです。

(6)「build.bat」

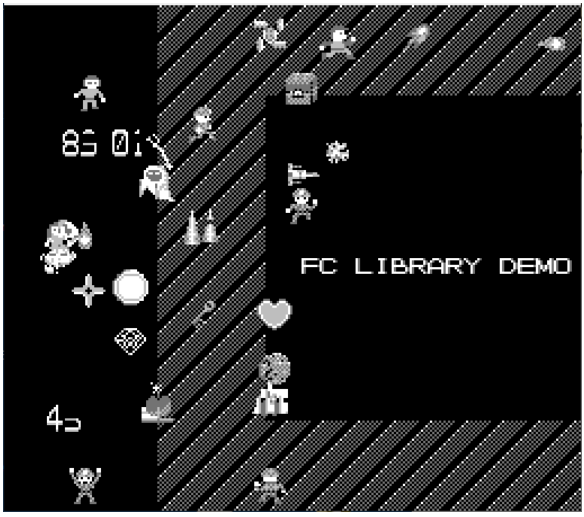
ビルド用のバッチファイルです。コンパイラとリンカを呼び出しています。長いコマンドは「^(キャレット)」で改行しています。

```
echo off
echo ROMイメージのビルドを行います
set SRC1=startup
set SRC2=main
set SRC3=fcsub
set CFGFILE=build.cfg
set COMPILE=C:\cc65\bin\cl65
set LINK=C:\cc65\bin\ld65
```

```
set LIBFILE=C:\cc65\lib\nes.lib
set NESFILE=rom.nes
%COMPILE% -t none -o %SRC1%.o -c %SRC1%.asm
if %errorlevel% neq 0 goto abort
%COMPILE% -t none -o %SRC2%.o -c -O %SRC2%.c
if %errorlevel% neq 0 goto abort
%COMPILE% -t none -o %SRC3%.o -c -O %SRC3%.c
if %errorlevel% neq 0 goto abort
%LINK% -o %NESFILE% --config %CFGFILE% ^
 --obj %SRC1%.o %SRC2%.o %SRC3%.o --lib %LIBFILE%
if %errorlevel% neq 0 goto abort
echo ビルドに成功しました
:abort
pause
```

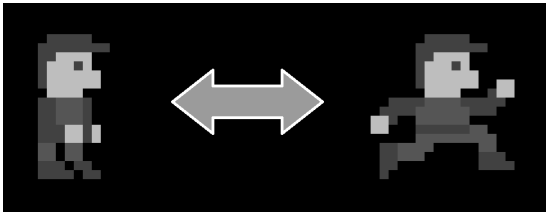


ファイルを用意できたら、「build.bat」を実行してビルドします。ビルドに成功すると、nesファイル（rom.nes）が出来上がります。

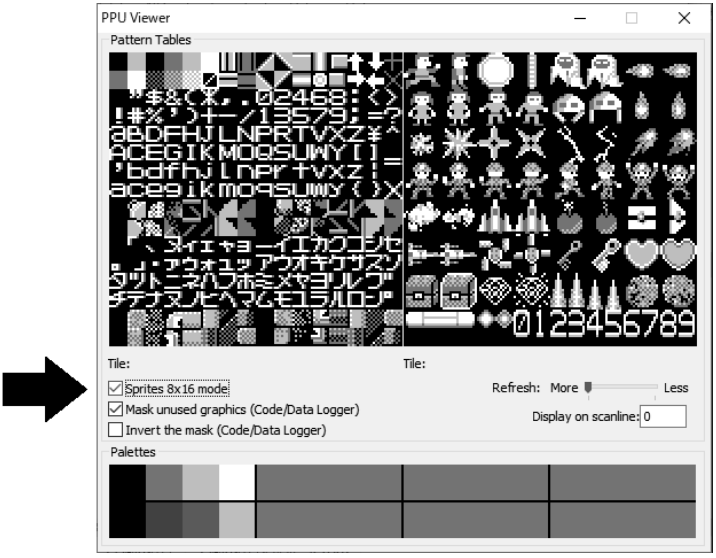


nesファイルの実行結果です。BGは横方向にスクロールします。スプラ

イトを使って、キャラクターを32体も表示しています。画面のあちこちでスプライトが消えてしまっています。残念ながらこれがファミコンの限界です。

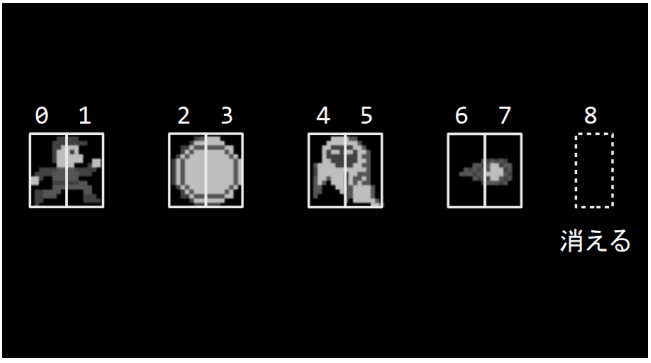


スプライトは2コマでアニメーションします。スプライトは「8×16モード」を使っています。「8×16モード」については後の章で紹介します。



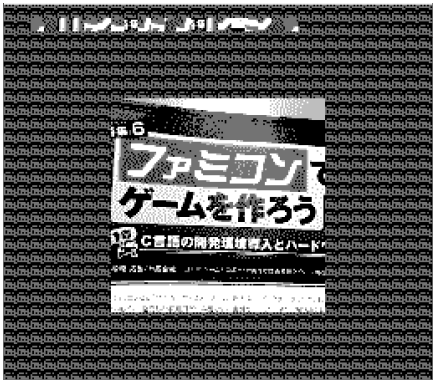
「FCEUX」の場合、メニューから「Debug」→ 「PPU Viewer」を選択して、「Sprites 8x16 mode」のチェックボックスをオンにすると、スプライトのタイル画像が8x16モードの配置で表示されます。ただし、その影響でBGのタイル画像の並びがおかしくなってしまいます。

《コラム》 スプライトが消える問題



ファミコンのスプライトは横方向に並んだ状態で表示できるのは「8枚まで」です。たとえば、上の図のようにスプライトを並べていくと、9枚目のスプライトが消えてしまいます。管理番号の大きいスプライトから消える仕組みです。

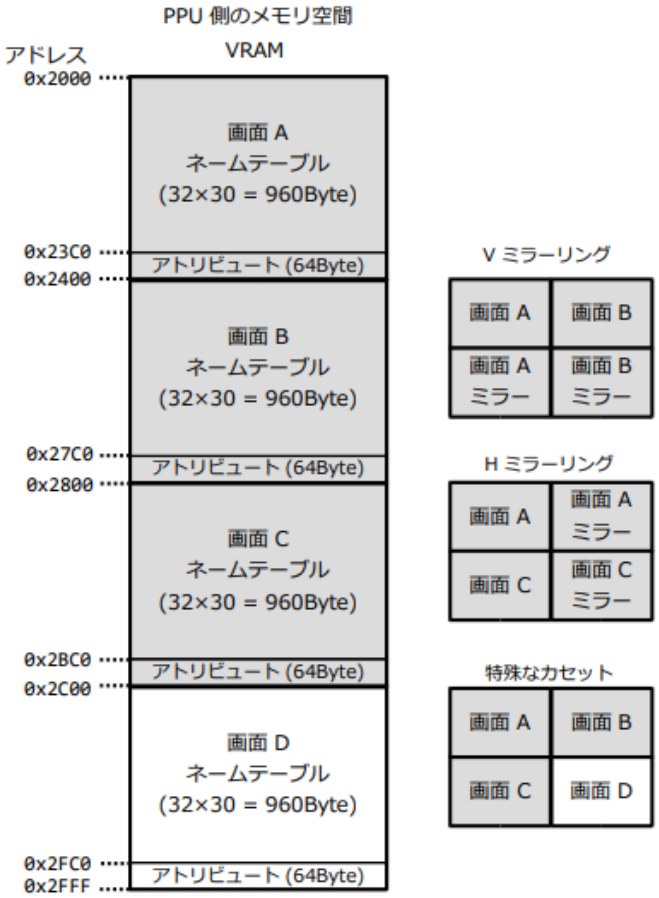
《コラム》 無理やりビットマップ表示



BGに16×16枚＝256枚のタイル画像を並べると、128×128ピクセルのビットマップを描くことができます。4KBも使っているのに、画像のサイズが小さいです。

第3章 BGを表示する

3-1 BGの仕様



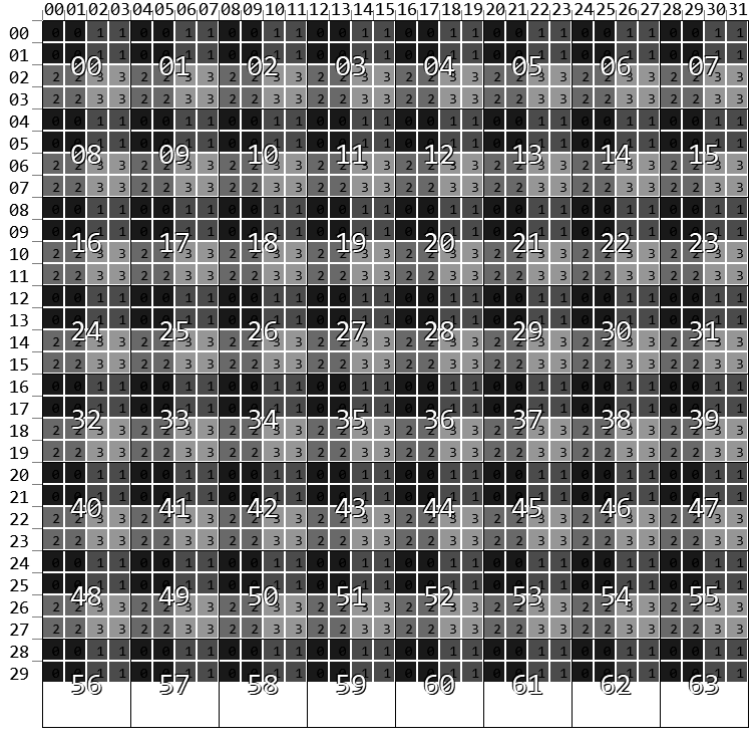
VRAMのメモリマップです。VRAMはPPU側のメモリ空間にあります。VRAMは「ネームテーブル」と「アトリビュートテーブル」を格納しています。「ネームテーブル」はBGのタイル番号を記憶するための領域です。「アトリビュートテーブル」はBGのパレット番号を記憶する領域です。

BGは2×2枚の画面が並んでいて、画面A～Dと呼ぶことにします。「Vミラーリング」の場合、画面Cは「画面Aのミラー（同じ内容）」となります。画面Dは「画面Bのミラー」になります。一方、「Hミラーリング」の場合、画面Bが「画面Aのミラー」で、画面Dは「画面Cのミラー」になります。ややこしいですが、画面が連続するのは、縦2画面か横2画面かのどちらかです。ミラーリングの設定はカセット内部の配線で決まります。通常はミラーリングを変更することができませんが、特別仕様のカセットは例外です。

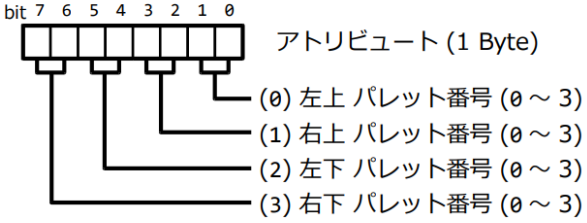
座標からアドレスを計算

画面を構成する最小単位を「セル」と呼ぶことにします。ネームテーブルは1セルのタイル番号を1バイト（0～255）で記憶します。1画面の容量は32×20セル＝960バイトです。ネームテーブルのアドレスは「画面の先頭アドレス+X座標+（Y座標×32）」の式で求めることもできます。たとえば、画面Aの座標（10,14）に表示するなら、0x2000+10+（14×32）＝0x21CA番地に書き込めばいいわけです。ネームテーブルへの書き込みはVブランク中に行うか、表示の停止中に行います。Vブランクの時間は短いので、一度に更新できる量には限界があります。FCライブラリではBGを描画する関数を用意してあります。たとえば、BGに1文字描画するには「bg_printch」関数を使います。詳しくは「fcsub.h」を参照してください。

3-2 BGのパレット



BGのパレット番号は「アトリビュートテーブル」に記憶します。上の図はアトリビュートテーブルのデータ構造です。記憶領域は64バイトあります。アトリビュートは常に2×2=4セル単位で設定します。1セル単位には設定できません。選択範囲の左上のX座標とY座標は必ず偶数になります。



アトリビュートのデータ構造です。2×2セルが4組集まって、1バイトのデータとなります。

BGのパレットを変更するプログラムです。2-4の「デモプログラム」の「main.c」を挿し替えて、ビルドしてください。

```
// Color Test for FC (main.c)
// by Takuya Matsubara
#include "fcsub.h"      // FCライブラリ
#include "pattern.h"    // パターンデータ

extern void waitvsync (void);  // Vブランク待ち

char palette[16];  // BG用パレット(4×4色)

//---- BG塗りつぶし
void bg_fill(char x,char y,char w,char h,char ch)
{
    char x1,y1;

    for (y1=0; y1<w; y1++){
        for (x1=0; x1<h; x1++){
            bg_printch(x+x1,y+y1,ch);
        }
    }
}

//---- スクリーン初期化
void screen_init(void)
{
    char x,y;
    char pal,i;
```

```
    unsigned char attr;
    char tempstr[6]="PAL._";

    ppu_enable(0);    //表示OFF
    // BGに描画
    for (pal=0; pal<4; pal++){
        x = pal*8;
        for (i=0; i<3; i++){
            // BG塗りつぶし4×4
            y = (i*8)+8;
            bg_fill(x,y,4,4,i+1);
            // BGアトリビュート設定
            attr = 0b01010101 * pal;
            bg_attrib(x,y,attr);
        }
        // パレット番号表示
        y = 4;
        tempstr[4] = '0'+pal;
        bg_printstr(x,y,tempstr);
    }
    waitvsync();      // Vblank待ち
    ppu_enable(1);    // 表示ON
}

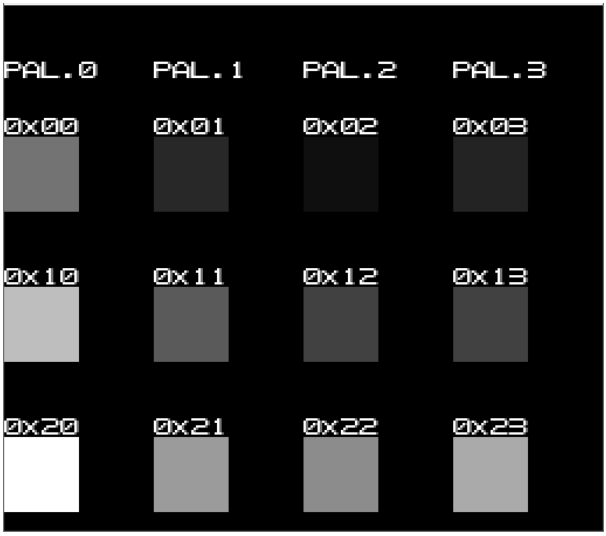
//---- パレット変更(色の種類,明るさのオフセット)
void palette_change(char htemp,char btemp)
{
    char i,pal,color;
    char x,y;
    char tempstr[5]="0x__";

    waitvsync();      // Vblank待ち
    ppu_enable(0);    // 表示OFF
    for(pal=0; pal<4; pal++){
        for(i=0; i<3; i++){
            color = (i*0x10)+htemp+btemp; // カラーコード
            palette[(pal*4)+i+1] = color; // パレットに格納
            x = pal*8;
            y = (i*8)+7;
            num_to_hex(tempstr+2,color); // 16進数表示
            bg_printstr(x,y,tempstr);
        }
        htemp = (htemp+1) % 13;  // 色の種類(0-12)
    }
    // パレット設定
    palette[0] = 0x0F;
    ppu_palette((char *)palette,PAL_0,4);
    waitvsync();      // Vblank待ち
```

```
    ppu_enable(1);    // 表示ON
}

// メイン
char NesMain()
{
    unsigned char loopcnt = 0;
    char hue = 0;     // 色の種類
    char bright = 0;  // 明るさ

    fc_init();   // 全機能を初期化
    ppu_pattern((unsigned char *)pattern,0,512);
    screen_init();               // BGを描画
    palette_change(hue,bright);  // パレット変更
    while (1){
        loopcnt++;
        if(loopcnt >= 180){ // 3秒経過
            loopcnt = 0;
            hue++;  // 色の種類を循環
            if(hue >= 13){
                hue = 0;
                bright ^= 0x10; // 明るさを変更
            }
            palette_change(hue,bright);
        }
        waitvsync();     // Vblank待ち
        bg_scroll(0, 0);
    }
    return 0;
}
```

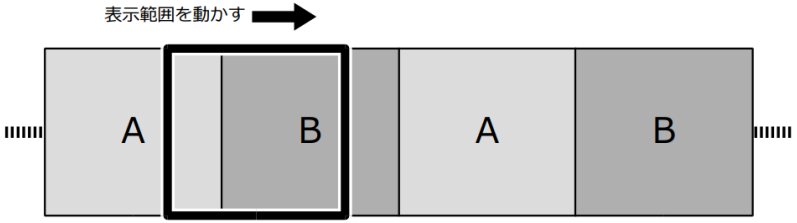


nesファイルの実行結果です。パレットを4つ使って、12色の四角形をBGに表示します。背景色も加えると13色です。パレットは3秒おきに自動的に切り替わります。全部のカラーコードを一度に表示できないので、0x00～0x2C番と0x10～3C番の範囲を交互に切り替えています。実際に実行してみて色彩の豊かさを感じてください。

《コラム》ファミコンの同時発色数

BG用とスプライト用を合計するとパレットは8つ。これに背景色を加えると、(8×3)+1＝25。ファミコンが同時に出せる色は最大で「25色」です。

3-3 BGのスクロール

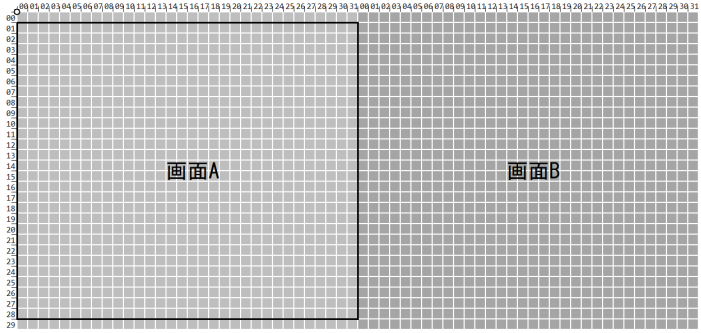


「Vミラーリング」の場合、BGの「画面A」と「画面B」は図のようにつながっています。BGをスクロール表示させるには、BGオフセット座標を変更します。X座標を変更すると、画面が横方向に動きます。

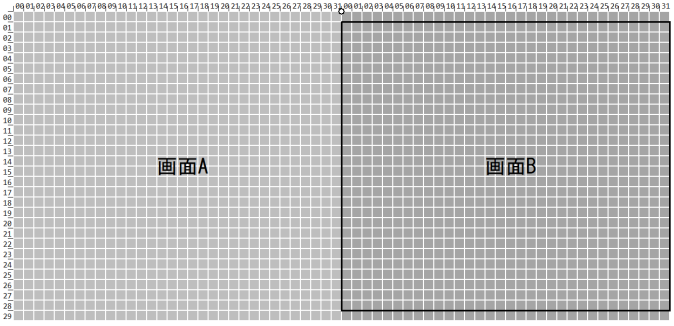
CPU 側のメモリ空間

アドレス		
0x2005	PPU スクロールレジスタ（ PPUSCROLL ）	* 書き込み専用 BG の表示オフセット (X 座標、 Y 座標の順に書き込む)
0x2006	PPU アドレスレジスタ（ PPUADDR ）	* 書き込み専用 PPU アドレス (アドレス上位、下位の順に書き込む)
0x2007	PPU データレジスタ（ PPUDATA ）	PPU を読み書きするレジスタ

BGオフセット座標を変更するには、「PPUスクロールレジスタ(PPUSCROLL)」というレジスタに書き込みます。1回目にX座標を書き込み、2回目にY座標を書き込みます。



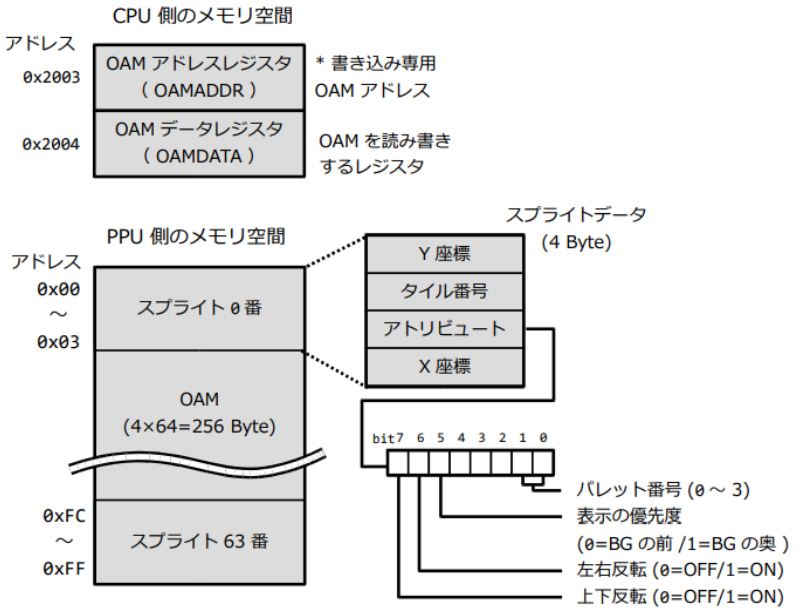
オフセット座標が(0,0)の場合、画面Aだけが表示されます。画面1枚のサイズは256×240ピクセル(32×30セル)ですが、256×224ピクセル(32×28セル)を切り抜いたように表示します。ブラウン管方式のテレビだと表示範囲はさらに狭くなります。



オフセット座標のX座標を増やすと、表示は右から左にスクロールします。仮にX座標が「255」ならば、1を足すと循環して「0」になります。その時、「PPU制御レジスタ(PPUCTRL)」のbit1～0を書き換えて、制御対象のネームテーブルを「画面B」に変更します。同じようにX座標が「0」に循環したら、制御対象を「画面A」に戻します。このように対象を交互に切り替えて、無限にスクロールさせることができます。

第4章 スプライトを表示する

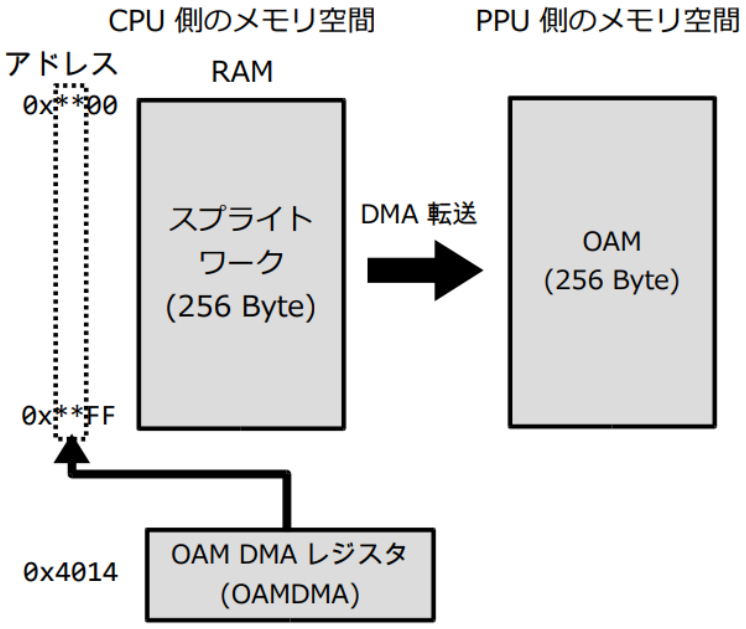
4-1 スプライトの仕様



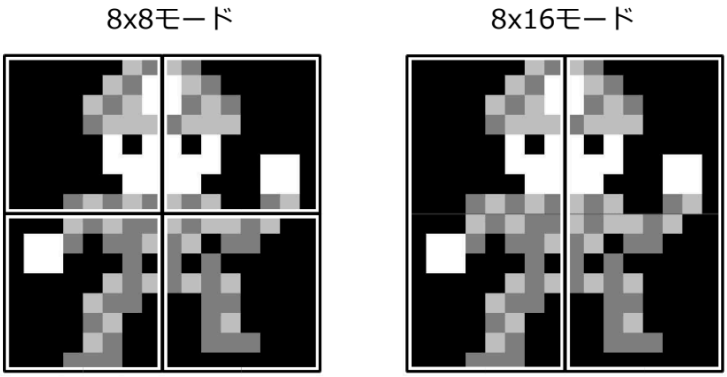
スプライトのデータはOAM(Object Attribute Memory)というPPUの中にある領域に格納されます。管理できるスプライトは64枚(0～63番)です。スプライトデータは「Y座標」「タイル番号」「アトリビュート」「X座標」の順に1枚あたり4バイトを格納しています。全部で4バイト×64枚=256バイトです。OAMは0x00～FF番地という独自のアドレスを持っています。CPU側のメモリ空間にある「OAMアドレスポート(OAMADDR

)」と「OAMデータポート(OAMDATA)」というレジスタを使えば、OAMにデータを書き込むことができます。しかし、Vブランク中か表示の停止中にしかアクセスできないので、使い勝手はイマイチです。

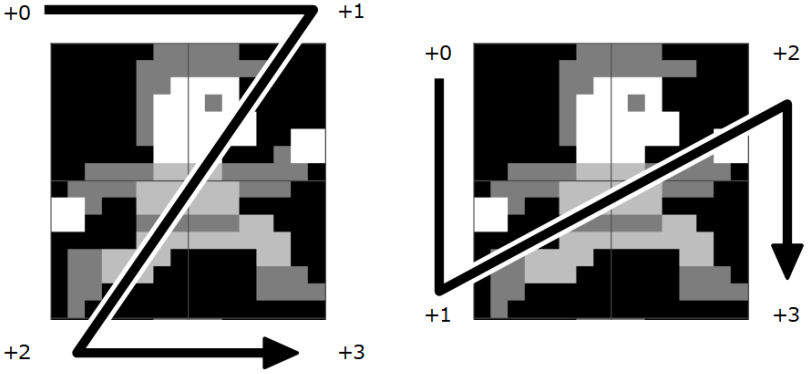
DMA転送



そこでDMA転送を使います。これを使うと、一度命令すればCPUとは無関係にOAMにデータを書き込んでくれます。ただし、RAMが256バイト必要でアドレスの下位が0x00～FF番地でないといけない決まりがあります。この領域のことをFCライブラリでは「スプライトワーク」と命名して、0x0700～07FF番地に割り当てています。DMA転送をしたいときは「sp_dmastart」関数を呼び出すだけです。グローバル変数を大量に作るとワークと衝突するので要注意です。

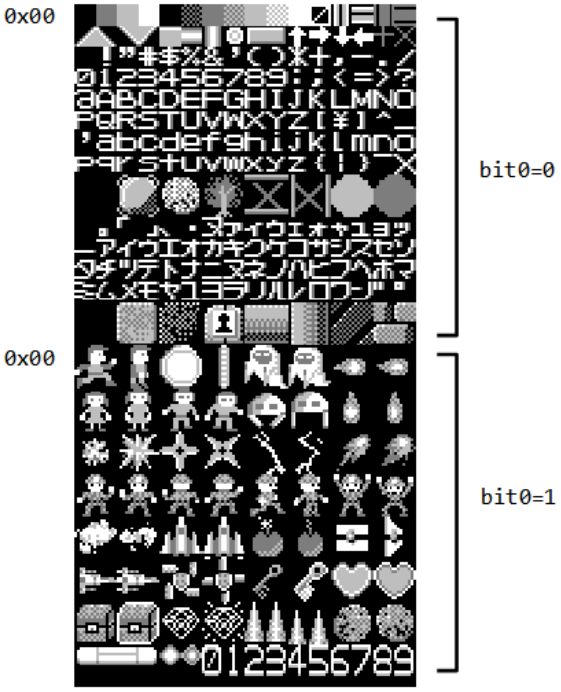


ファミコンのスプライトは「8×8モード」と「8×16モード」という2つのモードがあります。たとえば、16×16ピクセルのキャラクターを表示したい場合、8×8モードは4枚のスプライトが必要ですが、8×16モードは2枚のスプライトで済みます。8×16モードの方が2倍お得です。



8×16モードを使うルールとして、スプライトは「上半分」と「下半分」でタイル番号が連続してないといけません。スプライト2枚を連続させた場合はタイル画像の並びは「逆N型」になります。

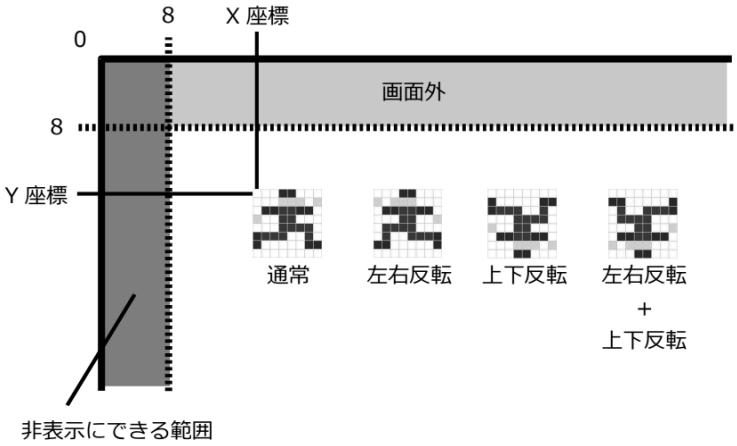
2-4で作成した「pattern.h」の配列はタイル画像を16枚ずつ「Z型」に並べて格納しているので、そのままでは8×16モードで使うことができません。この問題を解決するため、FCライブラリでは「ppu_pattern8x16」関数を作りました。この関数は「Z型」の配列を「逆N型」に並べ直してパターンテーブルに登録します。最初から配列を「逆N型」に格納することもできますが、それだと使いにくい場合があるので、この方法に落ち着きました。



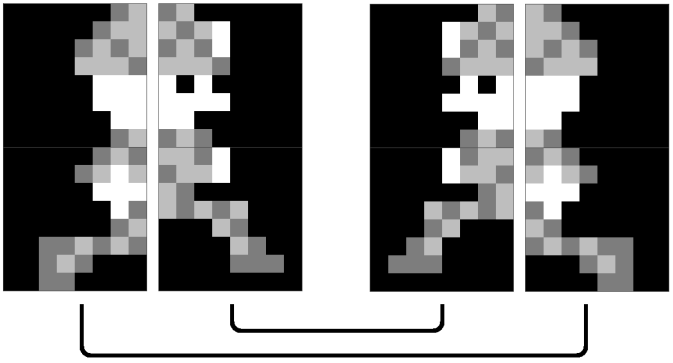
8×16モードでOAMに書き込む場合、タイル番号のbit0はオフセットフラグとなります。フラグが「0」の場合はアドレスのオフセットが0x0000番地、フラグが「1」の場合はアドレスのオフセットが0x1000番地と解釈されます。タイル番号は強制的に偶数として扱われるので、タイル番号+オフ

セットフラグという指定方法になります。複雑ですが、この仕様によって、タイル画像を512枚使うことができます。

4-2 スプライトのパレット



スプライトのパレットを変更するには、OAMの「アトリビュート」を変更します。アトリビュートは「パレット番号(0～3)」「左右反転」「上下反転」「優先度(BGより手前／BGより奥)」を1バイトにまとめたデータです。



8×16モードで図のようなキャラクターを左右反転した場合、左右の表示内容を入れ替える必要があります。しかし、スプライトの座標を入れ替えると処理が複雑になってしまいます。そこで、スプライトの座標はそのままにして、タイル画像だけを入れ替えます。具体的な処理方法はFCライブラリ(fcsub.c)の「sp_chr」関数を参照してください。

スプライトのアトリビュートを設定するプログラムです。2-4の「デモプログラム」の「main.c」を挿し替えて、ビルドしてください。

```
// スプライトのアトリビュート設定 for FC (main.c)
// by Takuya Matsubara
#include "fcsub.h"     // FCライブラリ
#include "pattern.h"   // パターンデータ

extern void waitvsync (void); // Vブランク待ち

const char palette[] = { // カラーパレット
    C_BLACK, C_BLUE  , C_RED   , C_YELLOW // SPパレット0
};

// メイン関数
char NesMain(void)
{
    unsigned char x,y;
    unsigned char sp1,sp2,sp3,sp4,sp5;
    fc_init();      // 全機能の初期化
    // パターンテーブル設定 &  カラーパレット設定
    ppu_pattern((unsigned char *)pattern,0,256);
    ppu_pattern8x16((unsigned char *)pattern+4096,256,256);
    ppu_palette((char *)palette,PAL_0+4,1);
    // BGに描画
    for(y=3; y<27; y++){
        for(x=2; x<30; x++){
            bg_printch(x,y,'Z');
        }
    }
    // スプライト追加
    sp1 = sp_append(64,64,0+1,PAL_0);

    // スプライト追加 左右反転
    sp2 = sp_append(96,64,0+1,PAL_0+ATR_FLIPH);

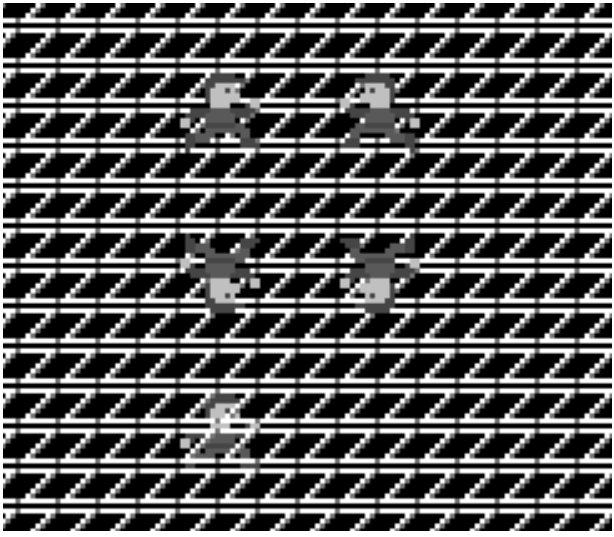
    // スプライト追加 上下反転
```

```
    sp3 = sp_append(64,96,0+1,PAL_0+ATR_FLIPV);

    // スプライト追加 左右反転+上下反転
    sp4 = sp_append(96,96,0+1,PAL_0+ATR_FLIPH+ATR_FLIPV);

    // スプライト追加 BGの奥に表示
    sp5 = sp_append(64,128,0+1,PAL_0+ATR_BEHIND);

    waitvsync();           // Vブランク待ち
    ppu_enable(1);         // 表示有効化
    while (1){
        waitvsync();       // Vブランク待ち
        sp_dmastart();     // OMA DMA転送開始
        bg_scroll(0,0);    // BGスクロール
    }
    return 0;
}
```



nesファイルの実行結果です。スプライトを使って、5体のキャラクターを表示します。見づらいですが、5体目のキャラクターは画面左下のBGの奥に居ます。アトリビュートを変えると、ここまで多彩な表現ができます。

4-3 スプライトの衝突

スプライトとスプライトの衝突を判定するプログラムです。2-4の「デモプログラム」の「main.c」を挿し替えて、ビルドしてください。

```
// スプライトの衝突判定 for FC (main.c)
// by Takuya Matsubara
#include "fcsub.h"     // FCライブラリ
#include "pattern.h"   // パターンデータ

extern void waitvsync (void); // Vブランク待ち

const char palette[] = { // カラーパレット
    C_BLACK, C_BLUE  , C_RED   , C_YELLOW, // SPパレット0
    0      , C_BLACK , C_GRAY1 , C_WHITE   // SPパレット1
};

#define THRESHOLD 15  // 衝突判定のしきい値

// メイン関数
char NesMain(void)
{
    unsigned char x,y,hit;
    char x1=-1;
    unsigned char player,enemy;
    char tmpstr[3];
    fc_init();      // 全機能の初期化
    // パターンテーブル設定
    ppu_pattern((unsigned char *)pattern,0,256);
    ppu_pattern8x16((unsigned char *)pattern+4096,256,256);
    ppu_palette((char *)palette,PAL_0+4,2);    // カラーパレット設定
    player = sp_append(64,96,0+1,PAL_0);    // プレイヤー追加
    enemy = sp_append(128,96,16+1,PAL_1);   // オバケ追加
    bg_printstr(8,20,"SPRITE HIT:");
    waitvsync();          // Vブランク待ち
    ppu_enable(1);
    while (1){
        x = sp_getx(player);
        y = sp_gety(player);
        x += x1;
        if((x<64)||(x>192)) x1 = -x1;
        sp_ofs(player,x,y); // スプライトの移動
        // スプライトの衝突判定
        hit = sp_hitcheck(enemy,x,y,THRESHOLD);
        num_to_hex(tmpstr,hit);
```

```
        waitvsync();            // Vブランク待ち
        sp_dmastart();          // OMA DMA転送開始
        bg_printstr(19,20,tmpstr);  // 衝突結果
        bg_scroll(0,0);         // BGスクロール
    }
    return 0;
}
```



nesファイルの実行結果です。スプライトは自動的に左右に動きます。画面の「SPRITE　HIT」は衝突判定の結果です。スプライトが衝突すると、値が「00」→「01」に変化します。PPUには0番のスプライトの衝突判定機能がありますが、今回は使っていません。判定には「sp_hitcheck」という自作の関数を使っています。ここでは、15ピクセル以内にスプライトがある場合、「衝突」と判定しています。判定のしきい値を変更したい場合はdefine文の「THRESHOLD」の値を書き換えます。

4-4 BGとの衝突

スプライトとBGの衝突を判定するプログラムです。2-4の「デモプログラム」の「main.c」を挿し替えて、ビルドしてください。

```
// スプライトとBGとの衝突 for FC (main.c)
// by Takuya Matsubara
#include "fcsub.h"     // FCライブラリ
#include "pattern.h"   // パターンデータ

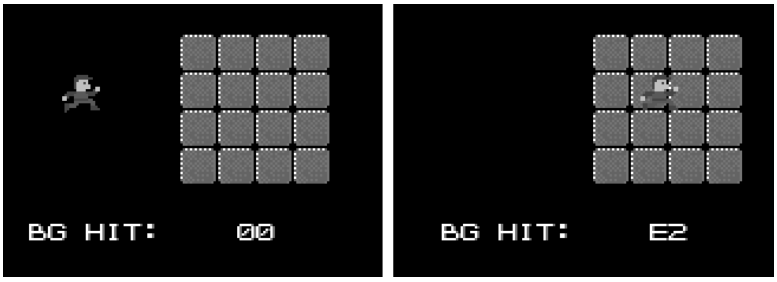
extern void waitvsync (void); // Vブランク待ち

const char palette_bg[] = { // カラーパレット
    C_BLACK, C_GRAY , C_GREEN , C_WHITE  // BGパレット0
};

const char palette_sp[] = { // カラーパレット
    C_BLACK, C_BLUE , C_RED   , C_YELLOW // SPパレット0
};

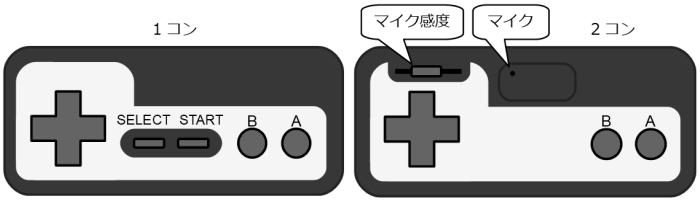
// メイン関数
char NesMain(void)
{
    unsigned char x,y,hit;
    char x1 = 1;
    unsigned char player;
    unsigned short adr;
    char tmpstr[3];
    fc_init();      // 全機能の初期化
    // パターンテーブル設定
    ppu_pattern((unsigned char *)pattern,0,256);
    ppu_pattern8x16((unsigned char *)pattern+4096,256,256);
    // カラーパレット設定
    ppu_palette((char *)palette_bg,PAL_0  ,1);
    ppu_palette((char *)palette_sp,PAL_0+4,1);
    // BGに描画
    bg_printstr(8,20,"BG HIT:");
    for(y=10; y<18; y+=2){
        for(x=16; x<24; x+=2){
            bg_print2x2(x,y,0xe2);
        }
    }
    // スプライト追加
    player = sp_append(64,96,0+1,PAL_0);
    waitvsync();          // Vブランク待ち
    ppu_enable(1);
    while (1){
        x = sp_getx(player);
        y = sp_gety(player);
        x += x1;
        if((x<64)||(x>192))x1 = -x1;
        sp_ofs(player,x,y); // スプライトの移動
        x /= 8;
```

```
        y /= 8;
        adr = 0x2000+x+((unsigned short)y*32);
        waitvsync();           // Vブランク待ち
        sp_dmastart();         // OMA DMA転送開始
        ppu_address( adr );
        hit = PPUDATA;         // ネームテーブルの読み取り
        num_to_hex(tmpstr,hit);
        bg_printstr(19,20,tmpstr);  // 衝突結果
        bg_scroll(0,0);        // BGスクロール
    }
    return 0;
}
```

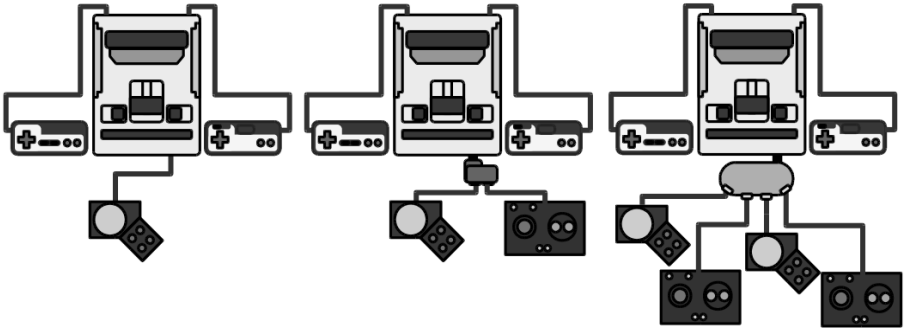


nesファイルの実行画面です。スプライトが自動的に動きます。画面の「BG　HIT」はスプライトが触れているタイル番号です。何も触れていない場合は「透明なタイル」に触れている状態なので、「00」が表示されます。ブロックに触れた場合、「E2」や「E3」と表示されます。ネームテーブルを読み込むのにPPUデータレジスタ（PPUDATA）を使っていますが、アクセスできるのがVブランク中か表示の停止中に限られているので、使い勝手がよくありません。障害物の情報を配列に入れて衝突判定した方が良さそうです。細かい話ですが、BGがスプライトよりも1ピクセル上に表示されてしまいます。これはファミコンの仕様だと思いますが、ぴったり重ならないのが気になります。BGオフセット座標のY座標を「239」に変更にすると、BGが1ピクセル下に移動して、ぴったり重なります。

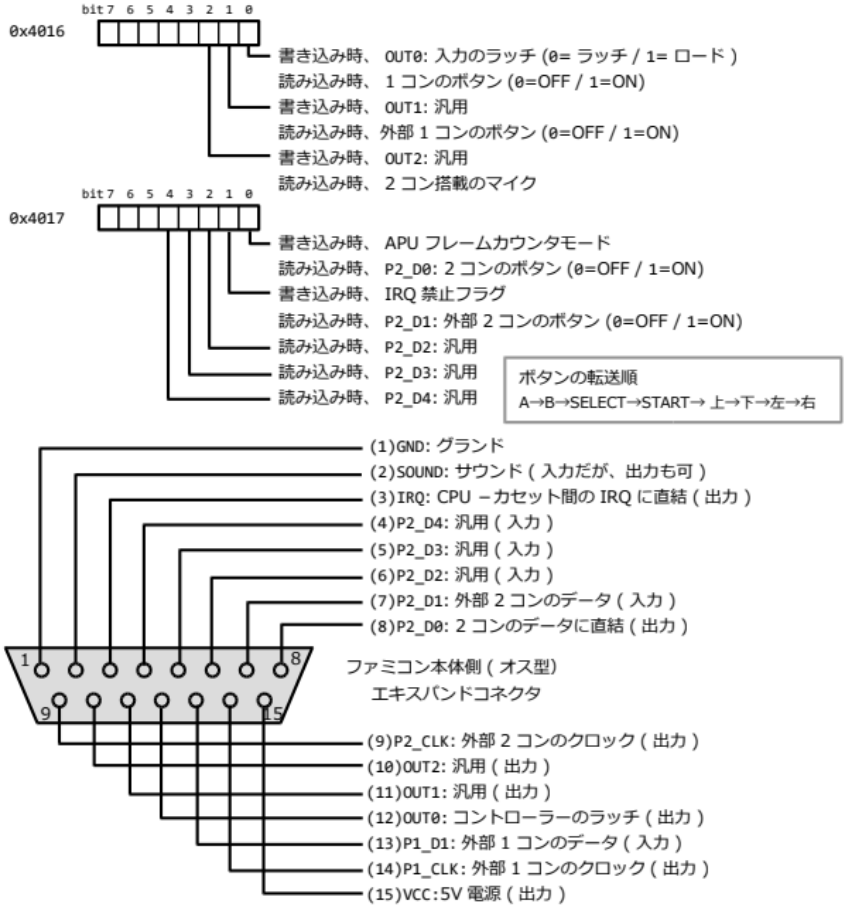
第5章 コントローラーの入力



コントローラーの入力方法を紹介します。ここでは、本体備え付けのコントローラーを「1コン」「2コン」と呼び、エキスパンドコネクタに接続するコントローラーを「外部1コン」「外部2コン」と呼ぶことにします。



外部コンの接続方法は大きく分けて3通りあります。まずはそのまま接続する方法です。次に2人用マルチタップ（HAL研究所「ジョイペア」やホリ「ツインアダプタ」など）を使って、外部コンを2つ接続する方法です。そして、4人用マルチタップ（ホリ「4プレイヤーズアダプター」）を使って、外部コンを4つ接続する方法があります。



コントローラーを入力したり、エキスパンドコネクタを制御するにはCPU側のメモリ空間にある0x4016～4017番地を読み書きします。ここでは、0x4016番地を「JOYPAD1」、0x4017番地を「JOYPAD2」と呼ぶことにします。

5-1 一人で遊ぶ

FCライブラリでは「1コン」と「外部1コン」のどちらを操作しても「同じ入力」として扱っています。市販のゲームでよく見られる入力方法です。1コン+外部1コンを読み取るプログラムの例は次の通りです。

```
unsigned char controller1(void)
{
    unsigned char ret=0;
    char i;
    JOYPAD1 = 0x01;    // ボタン入力を開始
    JOYPAD1 = 0x00;    // ボタン入力をラッチ
    for(i=0; i<8; i++){
        ret <<= 1;
        if( JOYPAD1 & 3) ret |= 1;  // ボタンが押された
    }
    return(ret);   // 戻り値=ボタンの状態(8bit)
}
```

JOYPAD1(0x4016番地)に1→0を書き込んでボタンの状態を固定(ラッチ)します。そして、JOYPAD1を8回読み取ります。読み取った値のbit0が「1」、またはbit1が「1」の場合は「ボタンがON」と判断します。関数の戻り値に対応するボタンは次の通りです。

・bit7：Aボタン

・bit6：Bボタン

・bit5：SELECTボタン

・bit4：STARTボタン

・bit3：十字ボタン上

・bit2：十字ボタン下

・bit1：十字ボタン左

・bit0：十字ボタン右

5-2 二人で遊ぶ

1コンと一緒に2コンを読み取るプログラムです。この関数を呼び出すと、グローバル変数の「btn1」に1コン+外部1コンのボタン、「btn2」に2コン+外部2コンのボタンの状態が格納されます。

```
unsigned char btn1; // 1コン+外部1コンのボタン
unsigned char btn2; // 2コン+外部2コンのボタン

void controller2(void)
{
    char i;
    JOYPAD1 = 0x01;    // ボタン入力を開始
    JOYPAD1 = 0x00;    // ボタン入力をラッチ
    for(i=0; i<8; i++){
        btn1 <<= 1;
        btn2 <<= 1;
        if( JOYPAD1 & 3) btn1 |= 1;  // 1コン+外部1コンのボタン
        if( JOYPAD2 & 3) btn2 |= 1;  // 2コン+外部2コンのボタン
    }
}
```

5-3 四人で遊ぶ、その1

1コン、2コン、外部1コン、外部2コンをそれぞれ「別の入力」として扱うことで、4人同時プレイを実現するプログラムです。エキスパンドコネクタに2人用マルチタップとコントローラー2つを接続する必要があります。関数を呼び出すと、グローバル変数の「btn1」に1コンのボタン、「btn2」に2コンのボタン、「btn3」に外部1コンのボタン、「btn4」に外部2コンのボタンが格納されます。

```
unsigned char btn1; // 1コンのボタン
unsigned char btn2; // 2コンのボタン
unsigned char btn3; // 外部1コンのボタン
unsigned char btn4; // 外部2コンのボタン
```

```
void controller4(void)
{
    char i,temp1,temp2;
    JOYPAD1 = 0x01;    // ボタン入力を開始
    JOYPAD1 = 0x00;    // ボタン入力をラッチ
    for(i=0; i<8; i++){
        btn1 <<= 1;
        btn2 <<= 1;
        btn3 <<= 1;
        btn4 <<= 1;
        temp1 = JOYPAD1;
        temp2 = JOYPAD2;
        if(temp1 & 1) btn1 |= 1;  // 1コンのボタン
        if(temp2 & 1) btn2 |= 1;  // 2コンのボタン
        if(temp1 & 2) btn3 |= 1;  // 外部1コンのボタン
        if(temp2 & 2) btn4 |= 1;  // 外部2コンのボタン
    }
}
```

5-4 四人で遊ぶ、その2

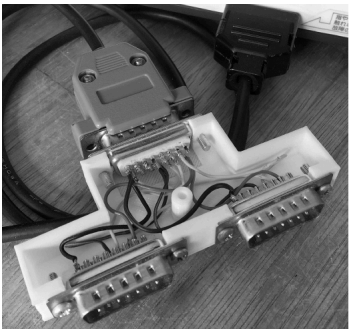
4人用マルチタップで4つの外部コンを読み取るプログラムです。エキスパンドコネクタにホリ「4プレイヤーズアダプター」と4つの外部コンを接続する必要があります。マルチタップのモードは「4Pモード」に設定します。ボタンのデータは8ビットを2回に分けて入力します。念のため、1コンと2コンを外部1コンと外部2コンと「同じ入力」として扱いました。もし、1コンと2コンを「別の入力」として扱えば、「6人同時プレイ」が可能です。

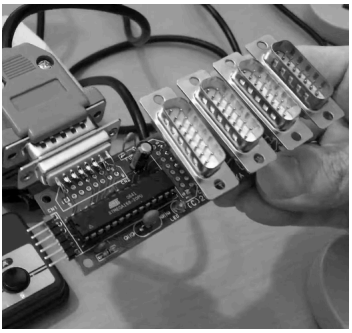
```
unsigned char btn1; // 1コン+外部1コンのボタン
unsigned char btn2; // 2コン+外部2コンのボタン
unsigned char btn3; // 外部3コンのボタン
unsigned char btn4; // 外部4コンのボタン

void controller4_hori(void)
{
    char i;
    JOYPAD1 = 0x01;    // ボタン入力をロード
    JOYPAD1 = 0x00;    // ボタン入力をラッチ
```

```
    for(i=0; i<8; i++){
        btn1 <<= 1;
        btn2 <<= 1;
        if( JOYPAD1 & 3) btn1 |= 1;  // 1コン+外部1コン
        if( JOYPAD2 & 3) btn2 |= 1;  // 2コン+外部2コン
    }
    for(i=0; i<8; i++){
        btn3 <<= 1;
        btn4 <<= 1;
        if( JOYPAD1 & 2) btn3 |= 1;  // 外部3コン
        if( JOYPAD2 & 2) btn4 |= 1;  // 外部4コン
    }
}
```

《コラム》 マルチタップの自作

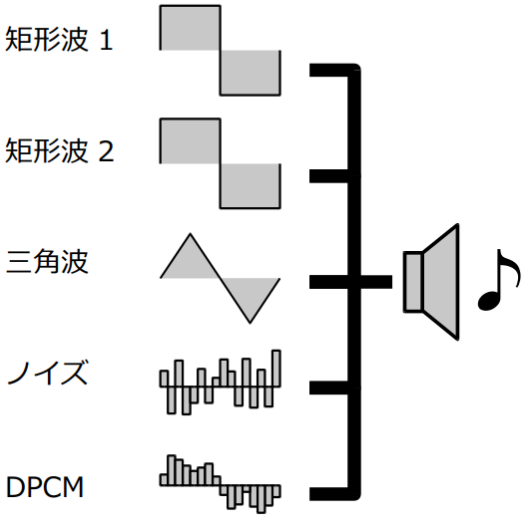




筆者が自作したマルチタップの資料をGitHubで公開しています。作る時は事故を起こさないようにご注意ください。

https://github.com/nicotakuya/diyfcmultitap

第6章 サウンド



APUは「矩形波1」「矩形波2」「三角波」「ノイズ」「DPCM」という、5つのチャンネルから音を同時に鳴らすことができます。各チャンネルは「サウンドチャンネル制御(SNDCHANNEL)」で有効／無効を設定します。

・0x4015番地：サウンドチャンネル制御(SNDCHANNEL)。
bit4:DPCM再生(0＝OFF / 1＝ON)。
bit3:ノイズ再生(0＝OFF / 1＝ON)。
bit2:三角波再生(0＝OFF / 1＝ON)。
bit1:矩形波2再生(0＝OFF / 1＝ON)。
bit0:矩形波1再生(0＝OFF / 1＝ON)。

6-1 矩形波

指定した周波数で矩形波の音を鳴らします。他にも、音量を調節できる機能、「エンベロープループ」という音を繰り返し鳴らす機能、「スイープ」という周波数を変化させる機能があります。

・0x4000番地： 矩形波1制御レジスタ1（PULSE1CTR1）。
bit7～6:デューティ比（0～3）。2で50%に設定。
bit5:エンベロープループ（0=不使用 / 1=使用＆長さカウンタ無効）。
bit4:ボリュームフラグ（0=ループ間隔 / 1=ボリューム）
bit3～0:
bit4=0の場合はループ間隔（0=早い～15=遅い）。
bit4=1の場合はボリューム（0～15）。

・0x4001番地： 矩形波1制御レジスタ2（PULSE1CTR2）。
bit7:スイープ 有効化（0＝OFF / 1＝ON）。
bit6～4:スイープ 時間（0～7）。
bit3:スイープ 音程（0=低音に変化 / 1=高音に変化）。
bit2～0:スイープ 変化量（0～7）。

・0x4002番地： 矩形波1周波数1（PULSE1FREQ1）。
bit7～0:周波数下位8ビット。
周波数の設定値は「((1790000÷16)÷周波数)-1」で算出できます。

・0x4003番地：矩形波1周波数2（PULSE1FREQ2）。
bit7～3:長さカウンタ（0～31）。

bit2～0:周波数上位3ビット。

・0x4004番地：矩形波2制御レジスタ1(PULSE2CTR1)。

・0x4005番地：矩形波2制御レジスタ2(PULSE2CTR2)。

・0x4006番地：矩形波2周波数1(PULSE2FREQ1)。

・0x4007番地：矩形波2周波数2(PULSE2FREQ2)。

矩形波2は矩形波1とほぼ同じ機能です。

矩形波で音を鳴らすプログラムです。2-4の「デモプログラム」の「main.c」を挿し替えて、ビルドしてください。

```
// pulse test for FC (main.c)
// by Takuya Matsubara
#include "fcsub.h"     // ライブラリ
#include "pattern.h"   // タイルセット

extern void waitvsync(void); // Vブランク待ち

#define INIT 0
#define RANGE 1
#define KETA 2
#define MENUMAX 9
const unsigned char btnname[MENUMAX][12] = {
    "  NOTE 60+:", // 0
    " SWEEP VAL:", // 1
    "SWP PERIOD:", // 2
    "SWP NEGATE:", // 3
    "  VOL FLAG:", // 4
    "VOL/ENV_DV:", // 5
    "  ENV LOOP:", // 6
    "LENGTH CNT:", // 7
    "      DUTY:"  // 8
};

// 初期値、レンジ、ケタ
const char menus[MENUMAX][3]={
    { 9,13,2}, // 0:note 60+(0～12)
    { 4, 8,1}, // 1:スイープ変化量(0～7)
    { 4, 8,1}, // 2:スイープ時間(0～7)
```

```
    { 1, 2,1}, // 3:スイープ音程(0～1)
    { 1, 2,1}, // 4:ボリュームフラグ(0～1)
    { 8,16,2}, // 5:音量/ループ間隔(0～15)
    { 1, 2,1}, // 6:エンベロープループ(0～1)
    { 0,32,2}, // 7:長さカウンタ(0～31)
    { 2, 4,1}  // 8:デューティ(0～3)
};

char arg[MENUMAX];

// 周波数テーブル
const unsigned short freq_table[] = {
    262,  // C (ド)   60
    277,  // C#       61
    294,  // D (レ)   62
    311,  // D#       63
    330,  // E (ミ)   64
    349,  // F (ファ) 65
    370,  // F#       66
    392,  // G (ソ)   67
    415,  // G#       68
    440,  // A (ラ)   69
    466,  // A#       70
    494,  // B (シ)   71
    523   // C (ド)   72
};

//---- パラメータのデクリメント
char dec_value(char value,char range)
{
    if(value == 0) return(range-1);

    return(value -1);
}

//---- メイン関数
char NesMain()
{
    unsigned short hz,tmpfreq;
    char enveloop,length,duty;
    char volflag, vol_div;
    char sw_enable, sw_value, sw_period, sw_negate;
    char idx=0,idx_old=1;
    char i;
    unsigned char button,button_old=0;
    char tempstr[6];

    fc_init(); // 全機能の初期化
```

```
    // パターン設定 & BG描画
    ppu_pattern((unsigned char *)pattern,0,512);
    bg_printstr(2,2,(char *)"PULSE TEST  [A]PLAY");
    for(i=0; i<MENUMAX; i++){
        bg_printstr(5,4+i,(char *)btnname[i]);
        arg[i] = menus[i][INIT];
    }
    waitvsync();
    ppu_enable(1);    // 画面表示を有効化
    while (1){
        button = controller1();   // コントローラー1入力
        if(button != button_old){
            button_old = button;
            switch(button){
            case BUTTON_UP:
                idx = dec_value(idx,MENUMAX);
                break;
            case BUTTON_DOWN:
                idx = (idx + 1) % MENUMAX;
                break;
            case BUTTON_RIGHT:
                arg[idx] = (arg[idx]+1) % menus[idx][RANGE];
                break;
            case BUTTON_LEFT:
                arg[idx] = dec_value(arg[idx],menus[idx][RANGE]);
                break;
            case BUTTON_A:    // 再生
                hz = freq_table[arg[0]];     // 周波数
                sw_value = arg[1];  // スイープ変化量(0～7)
                sw_period = arg[2]; // スイープ時間(0～7)
                sw_negate = arg[3]; // スイープ音程(0=低音化/1=高音化)
                volflag = arg[4];   // ボリュームフラグ(0=ループ間隔
/1=ボリューム)
                vol_div = arg[5];   // ループ間隔/ボリューム(0～15)
                enveloop = arg[6];  // エンベロープループ(0=OFF/1=ON
＆長さカウンタ無効)
                length = arg[7];    // 長さカウンタ(0～31)
                duty = arg[8];      // デューティ比(0～3)
                if(sw_value==0){
                    sw_enable = 0;  // スイープOFF
                }else{
                    sw_enable = 1;  // スイープON
                }
                // 矩形波1制御レジスタ1,2
                PULSE1CTR1 = (duty<<6) + (enveloop<<5) +
(volflag<<4) + vol_div;
                PULSE1CTR2 = (sw_enable<<7) + (sw_period<<4) +
(sw_negate<<3) + sw_value;
```

```
                // 矩形波1周波数1,2
                tmpfreq = (unsigned short)((1790000/16)/hz)-1;
                PULSE1FREQ1 = tmpfreq & 0xff;
                PULSE1FREQ2 = (tmpfreq >> 8)+(length << 3);
                break;
            }
        }
        i = (i+1) % MENUMAX;
        num_to_str(tempstr,arg[i],menus[i][KETA]);
        waitvsync();     // Vブランク待ち
        bg_printstr(19,4+i,tempstr);
        if(idx_old != idx){
            bg_printch(3,4+idx_old,' ');
            idx_old = idx;
            bg_printch(3,4+idx    ,'>');
        }
        bg_scroll(0,0);  // BGスクロール
    }
    return 0;
}
```



nesファイルの実行結果です。十字ボタンで矩形波1のパラメータを変更できます。Aボタンを押すと音を鳴らします。「NOTE」はノート番号です。たとえば、69番だと440Hzの音を出します。

6-2 三角波

三角波の音を鳴らします。音量の調節ができません。スイープ機能もありません。

・0x4008番地：三角波制御レジスタ（TRIWAVCTR）。
bit7: カウンタ制御（0＝カウントダウン / 1＝長さカウンタ停止）。
bit6～0:リニアカウンタ（0～127）。

・0x400A番地：三角波周波数1（TRIWAVFREQ1）。
bit7～0:周波数下位8ビット。
周波数の設定値は「((1790000÷32)÷周波数)-1」で算出できます。

・0x400B番地：三角波周波数2（TRIWAVFREQ2）。
bit7～3:長さカウンタ（0～31）。
bit2～0: 周波数上位3ビット。

6-3 ノイズ

疑似乱数の音やブザー音を鳴らします。音量や周波数や再生間隔を指定できます。

・0x400C番地：ノイズ制御レジスタ（NOISECTR）。
bit5:エンベロープループ（0＝1回だけ再生 / 1＝ループ再生）
bit4:ボリュームフラグ（0＝ループ間隔 / 1＝ボリューム）
bit3～0:

bit4=0の場合はループ間隔(0=早い～15=遅い)。
bit4=1の場合はボリューム(0～15)。

・0x400E番地:ノイズ音質(NOISETONE)。
bit7:ノイズ モード(0=疑似乱数の音 / 1=ブザー音)。
bit3～0: ノイズ 周波数(0=高音 ～ 15=低音)。

・0x400F番地:ノイズ時間(NOISETIM)。
bit7～3:長さカウンタ(0～31)。

6-4 DPCM

DPCMはDifferential Pulse Code Modulationの略です。サンプリング音を鳴らすことができる機能です。NesDev.orgのWikiでは「DMC(Delta Modulation Channel:デルタ変調)と表記しています。DPCMは一般的なPCMと違って、加算か減算かの切り替えだけで波形を作っているので、音質が良くありません。大量にデータを消費するので、長時間鳴らすことが難しいです。

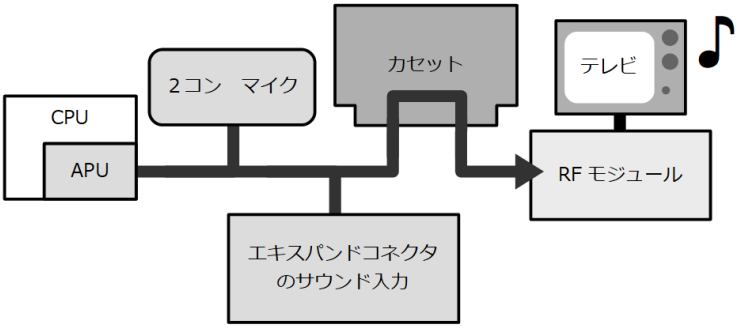
・0x4010番地:DPCM制御。
bit7:IRQ(0=無効 / 1=有効)。
bit6:ループ再生(0=OFF / 1=ON)。
bit3～0:サンプルレート(0=早い～15=遅い)。

・0x4011番地: DPCMデータ。bit0は無視されるので実質7ビット。

・0x4012番地：DPCMアドレス。アドレス＝0xC000 +（n×64）。

・0x4013番地：DPCM長さ。長さ = n×16 + 1 バイト。

《コラム》 サウンドのミキシング

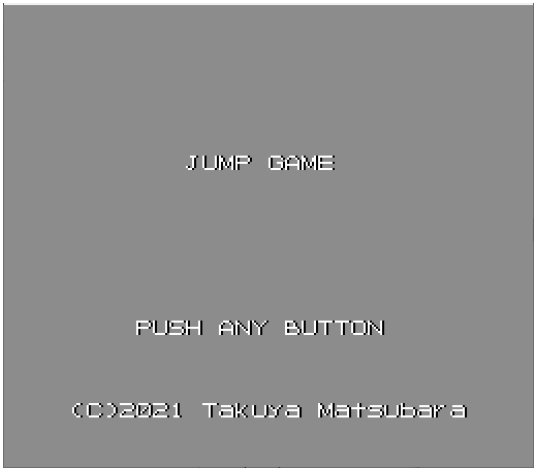


ファミコンのサウンドの信号がはAPUから出力されます。そこから2コンのマイクと混ざり、エキスパンドコネクタのサウンド入力（出力も可能）と混ざり、カセットを経由して、RFモジュールに届いて、テレビから音が鳴ります。

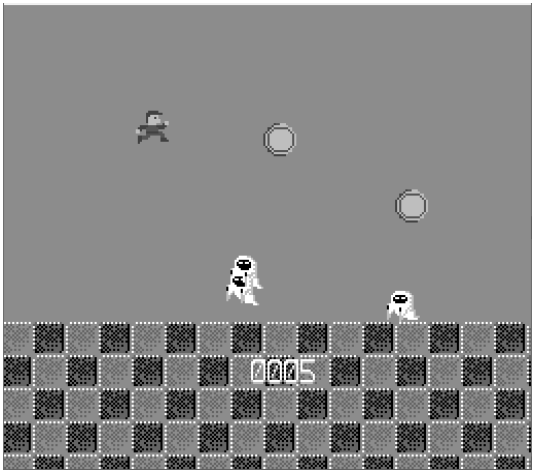


第7章 ファミコンゲームを作る

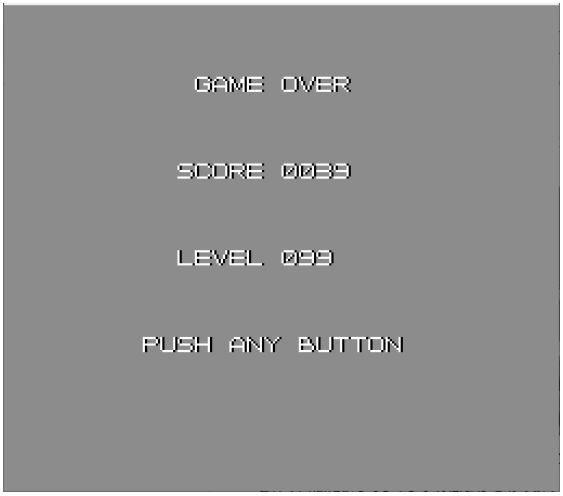
7-1 ジャンプゲーム



プレイヤーを操って、オバケを避けながらコインを集めるゲームです。



操作方法は十字ボタンの左右で移動。Aボタンでジャンプです。Aボタンを押す時間によってジャンプの高さが変わります。画面下の数字はスコア(コインを取った数)です。プレイ時間が経つと、ゲームレベル(オバケの出現率)が上がっていきます。



プレイヤーがオバケに触れたらゲームオーバーです。最後にスコアとゲームレベルが表示されます。

「Jump Game」のプログラムです。2-4の「デモプログラム」の「main.c」を挿し替えて、ビルドしてください。

```
// Jump Game V1.2 for FC (main.c)
// by Takuya Matsubara
#include "fcsub.h"     // FCライブラリ
#include "pattern.h"   // パターンデータ

extern void waitvsync (void); // Vブランク待ち

void pushbutton(void);
void title(void);
void result(void);
void player(char num,unsigned char x,unsigned char y);
void coin(char num,unsigned char x,unsigned char y);
void monster(char num,unsigned char x,unsigned char y);
```

```
char NesMain(void);

const char palette_bg[] = { // カラーテーブル
    C_SKYBLUE, C_BLACK , C_GREEN, C_WHITE  // BGパレット0
};

const char palette_sp[] = { // カラーテーブル
    C_SKYBLUE,C_BLUE  ,C_RED     ,C_YELLOW, // SPパレット0
    0         ,C_BLACK ,C_MAGENTA1,C_WHITE   // SPパレット1
};

const char strtitle[] = "JUMP GAME";
const char strcopyright[] = "(C)2021 Takuya Matsubara";
const char strpush[]  = "PUSH ANY BUTTON";
const char strgover[] = "GAME OVER";
const char strscore[] = "SCORE";
const char strlevel[] = "LEVEL";

// グローバル変数
short score;             // スコア
unsigned char gameover; // ゲームオーバーカウンタ
signed char jump;        // プレイヤーのジャンプ移動量
unsigned short bgx;      // BGオフセット座標
unsigned char loopcnt;  // ループカウンタ
unsigned char level;    // 難易度（敵の出現度数）

// タイル番号+(0x1000以降は+1)
#define TILE_MAN   (0x00+1)
#define TILE_COIN  (0x08+1)
#define TILE_ENEMY (0x10+1)

// ボタンの入力待ち
void pushbutton(void)
{
    unsigned char newbtn,oldbtn,edge;

    bg_printstr(8,20,(char *)strpush);   // PUSH
    waitvsync();      // Vブランク待ち
    sp_dmastart();    // OMA DMA転送開始
    ppu_enable(1);    // 表示を有効化
    oldbtn = 0;
    edge = 0;
    while(edge < 2){
        newbtn = controller1();   // コントローラー1入力
        if(oldbtn != newbtn) edge++;  // ボタンのエッジ検出
        oldbtn = newbtn;
        waitvsync();     // Vブランク待ち
        bg_scroll(0,0xef);  // BGスクロール
```

```
    }
    ppu_enable(0);   // 表示を無効化
    pulse1(300);     // 矩形波1サウンド再生
    bg_cls();        // BGを全消去
    sp_cls();        // スプライトを全消去
}

// タイトル＆ゲーム開始
void title(void)
{
    char x,y,tile;

    bg_printstr(11,10,(char *)strtitle);  // タイトル
    bg_printstr(4,25,(char *)strcopyright);
    pushbutton();    // ボタンの入力待ち
    for(y=20; y<30; y+=2){
        for(x=0; x<32; x+=2){
            tile = 226+((x+y)% 4);
            bg_print2x2(x,y   ,tile); // BG Aに描画
            bg_print2x2(x,y+32,tile); // BG Bに描画
        }
    }
    sp_append(64,120, TILE_MAN,PAL_0); // プレイヤーを追加
    score = 0;     // スコア
    jump = 0;      // ジャンプ移動量
    gameover = 0;  // ゲームオーバーカウンタ
    bgx = 0;       // BGオフセット座標
    loopcnt = 0;   // ループカウンタ
    level = 50;    // 難易度
    ppu_enable(1); // 表示を有効化
}

// リザルト画面
void result(void)
{
    char tmpstr[6];

    ppu_enable(0);    // 表示を無効化
    bg_cls();         // BGを全消去
    sp_cls();         // スプライトを全消去
    bg_printstr(11,5,(char *)strgover);  // GAME OVER
    bg_printstr(10,10,(char *)strscore); // SCORE表示
    bg_printstr(10,15,(char *)strlevel); // 難易度表示
    num_to_str(tmpstr,score,4);  // スコアを文字列化
    bg_printstr(16,10,tmpstr);   // スコア表示
    num_to_str(tmpstr,level,3);  // 難易度を文字列化
    bg_printstr(16,15,tmpstr);   // 難易度表示
    pushbutton();    // ボタンの入力待ち
```

```
}

// プレイヤー移動処理
void player(char num,unsigned char x,unsigned char y)
{
    unsigned char btn;     // ボタン情報

    if(gameover){
        sp_chr(0,TILE_MAN,ATR_FLIPH + ATR_FLIPV);
        if(y < OUTSIDE)y += 2;
        sp_ofs(num,x,y);
        return;
    }
    if((loopcnt % 32)==num)sp_anime(num);
    btn = controller1();   // コントローラー1入力
    if(((btn & BUTTON_LEFT ) != 0)&&(x > 8  )) x -= 2;  // 左
    if(((btn & BUTTON_RIGHT) != 0)&&(x < 228)) x += 2;  // 右
    if((y >= 144)&&((btn & BUTTON_A)!=0)) jump = -31;   // ジャンプ
    if(jump < 31){
        jump++;                          // 重力による加速
        if((btn & BUTTON_A)==0)jump+=2;  // Aボタンを押していない場合
    }
    y += (jump/4);
    if(y > 144) y = 144; // 地面のめり込み防止
    sp_ofs(num,x,y);
}

// コイン移動処理
void coin(char num,unsigned char x,unsigned char y)
{
    if((loopcnt % 32)==num)sp_anime(num);
    x--;
    if((gameover==0)&&(sp_hitcheck(0,x,y,15)==1)){ //プレイヤーと衝突
        y = OUTSIDE;  // コインを画面外へ
        pulse1(500);  // 矩形波1サウンド再生
        score++;      // スコアを加算
    }
    sp_ofs(num,x,y);  // スプライトの移動
}

// オバケ移動処理
void monster(char num,unsigned char x,unsigned char y)
{
    if((loopcnt % 32)==num)sp_anime(num);
    x--;
    x-=(num % 3);
    sp_ofs(num,x,y);
    if((gameover==0)&&(sp_hitcheck(0,x,y,10)==1)){ //プレイヤーと衝突
```

```
        pulse1(250);     // 矩形波1サウンド再生
        gameover = 1;    // ゲームオーバー
    }
}

// スコア表示 60～63番のスプライトを使用
void num_to_sp(unsigned short num)
{
    char i;
    unsigned char *ptr;
    char x,y;
    x = 144;
    y = 178;
    ptr = (unsigned char *)SPWADDR+(60*4); // スプライトワーク
    for(i=0; i<4; i++){
        *ptr++ = y;  // Y座標
        *ptr++ = ((num % 10)*2)+0xec+1; // タイル番号
        *ptr++ = 1;  // アトリビュート
        *ptr++ = x;  // X座標
        num /= 10;   // 1つ上のケタへ
        x -= 8;
    }
}

// メイン関数
char NesMain(void)
{
    unsigned char i,x,y;

    fc_init();      // 全機能の初期化
    // パターンテーブル & カラーパレット設定
    ppu_pattern((unsigned char *)pattern,0,256);
    ppu_pattern8x16((unsigned char *)pattern+4096,256,256);
    ppu_palette((char *)palette_bg,PAL_0  ,1);
    ppu_palette((char *)palette_sp,PAL_0+4,2);
    while (1){
       title();   // タイトル＆ゲーム開始
        while (gameover < 180){
            for(i=0; i<32; i++){
                y = sp_gety(i);
                if(y==OUTSIDE)continue; // 画面外のスプライト
                x = sp_getx(i);
                if(x <= 1){
                    sp_remove(i); // 画面外は消去
                    continue;
                }
                switch(sp_gettile(i)/8){
                case 0:
```

```
                    player(i,x,y);  // プレイヤー
                    break;
                case 1:
                    coin(i,x,y);    // コイン
                    break;
                case 2:
                    monster(i,x,y); // オバケ
                    break;
                }
            }
            loopcnt++;
            if((loopcnt & 63)==0){
                y = (random() % 8)*16 + 32;
                sp_append(240,y,TILE_COIN,PAL_0); // コイン出現
                if(level < 255)level++;  // 難易度を上げる
            }
            if((loopcnt & 15)==0 ){
                num_to_sp(score);        //スコア表示
                if((random() & 255) <= level){
                    y = (random() % 17)*8 + 16;
                    sp_append(240,y,TILE_ENEMY,PAL_1); // オバケ出現
                }
            }
            if(gameover)gameover++;
            bgx = (bgx+1) & 511;  // BGオフセット座標
            waitvsync();          // Vブランク待ち
            sp_dmastart();        // OMA DMA転送開始
            bg_scroll(bgx,0xef);  // BGスクロール
        }
        result();   // リザルト
    }
    return 0;
}
```

《コラム》 日経ソフトウエアのファミコン特集



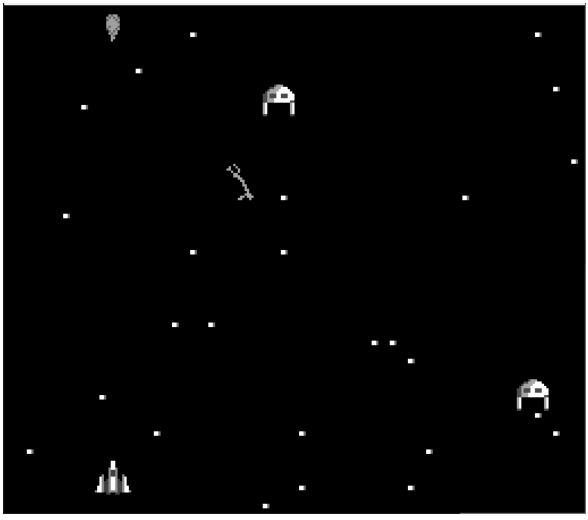
現在、「日経ソフトウエア」のファミコン特集の掲載号は「駿河屋」では買い取ってくれません。「0円」よりも悪い扱いで大ショックです。中古市場には出回らないとなると、Kindleで読むしかありません。雑誌版の「Jump Game」と本書のバージョンとの違いは次の通りです。

・スプライトの制御を8×8モード→8×16モードに変更。・フォントや床面の模様を変更。・プレイ中のスコア表示を追加。・ミスした時にプレイヤーが逆さまに表示される処理を追加。・コインやオバケの出現場所を変更。・リザルト画面にゲームレベルの表示を追加。

7-2 シューティングゲーム



シューティングゲームです。自機を操って敵を倒します。

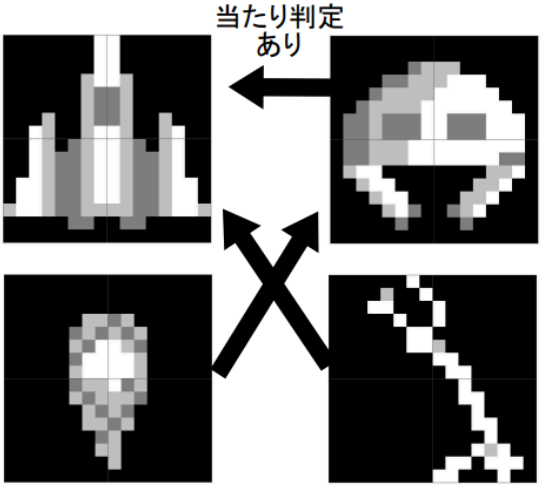


操作方法は十字ボタンで移動。Aボタンで弾の発射です。プレイ時間が進むと、自動的に難易度が上がっていきます。敵の数が増えて、敵の攻撃も

激しくなります。



敵や敵の弾に当たったらゲームオーバーです。リザルト画面ではスコア(SCORE)と難易度(LEVEL)が表示されます。



プログラムの仕組みを少し解説します。当たり判定の対象はキャラクターの種類によって違います。「自機」は「敵」と「敵の弾」に対して当たり判定があります。「自機の弾」は「敵」に対してのみ当たり判定があります。

本来は当たり判定用の配列を用意すべきですが、このプログラムではコードを節約するため、パレット番号で当たり判定のあり／なしを振り分けています。

・自機：パレット0番。

・自機の弾：パレット1番。

・敵の弾：パレット2番。

・敵：パレット3番。

パレットが2番以上かを比較すれば「敵または敵の弾」か「それ以外」かがわかります。パレットが3番以上かを比較すれば「敵」か「それ以外」かがわかります。これを当たり判定に応用しています。

「シューティングゲーム」のプログラムです。2-4の「デモプログラム」の「main.c」を挿し替えて、ビルドしてください。

```
// Jimi Shooting game(main.c) for FC
// by Takuya Matsubara
#include "fcsub.h"     // ファミコン用ライブラリ
#include "pattern.h"   // パターンデータ

extern void waitvsync (void); // Vブランク待ち

void pushbutton(void);
void title(void);
void result(void);
void player(char num,unsigned char x,unsigned char y);
void bakuhatu(char num,unsigned char x,unsigned char y);
void beam(char num,unsigned char x,unsigned char y);
void myshot(char num,unsigned char x,unsigned char y);
void enemy(char num,unsigned char x,unsigned char y);
char NesMain(void);

const char palette[] = { // カラーテーブル
    C_BLACK,C_BLUE    ,C_BLUE2   ,C_WHITE  , // SPパレット0
    0      ,C_GREEN1  ,C_GREEN2  ,C_TEAL2  , // SPパレット1
    0      ,C_MAGENTA1,C_PURPLE2 ,C_MAGENTA2,// SPパレット2
    0      ,C_PURPLE1 ,C_MAGENTA2,C_WHITE    // SPパレット3
};
```

```
const char strtitle[] = "JIMI SHOOTING GAME";
const char strcopyright[] = "(C)2026 Takuya Matsubara";
const char strpush[]  = "PUSH ANY BUTTON";
const char strgover[] = "GAME OVER";
const char strscore[] = "SCORE";
const char strlevel[] = "LEVEL";

// グローバル変数
short score;             // スコア
unsigned char gameover; // ゲームオーバーカウンタ
unsigned char bgy;       // BGオフセット座標
unsigned char loopcnt;  // ループカウンタ
unsigned char level;    // 難易度（敵の出現度数）
unsigned char tempdata[32];  // 汎用カウンタ

// タイル番号+（0x1000以降は+1）
#define TILE_SHIP   (0x88+1)
#define TILE_MYSHOT (0x38+1)
#define TILE_BEAM   (0x50+1)
#define TILE_ENEMY  (0x30+1)
#define TILE_BAKUHA (0x40+1)

// ボタンの入力待ち
void pushbutton(void)
{
    unsigned char newbtn,oldbtn,edge;
    bg_printstr(8,20,(char *)strpush);   // PUSH
    waitvsync();       // Vブランク待ち
    sp_dmastart();     // OMA DMA転送開始
    bg_scroll(0,0);    // BGスクロール
    ppu_enable(1);     // 表示を有効化
    oldbtn = 0;
    edge = 0;
    while(edge < 2){
        newbtn = controller1();   // コントローラー1入力
        if(oldbtn != newbtn) edge++;  // ボタンのエッジ検出
        oldbtn = newbtn;
        waitvsync();      // Vブランク待ち
        bg_scroll(0,0);  // BGスクロール
    }
    ppu_enable(0);     // 表示を無効化
    pulse1(300);       // 矩形波1サウンド再生
    bg_cls();          // BGを全消去
    sp_cls();          // スプライトを全消去
}

// タイトル＆ゲーム開始
void title(void)
```

```
{
    char x,y,i;
    bg_printstr(6,10,(char *)strtitle);     // タイトル
    bg_printstr(4,25,(char *)strcopyright); // Copyright
    pushbutton();    // ボタンの入力待ち
    for(i=0; i<30; i++){ // 星
        x = random() % 32;
        y = random() % 30;
        bg_printch(x,y,'.');
    }
    sp_append(128-8,240*2/3, TILE_SHIP,PAL_0); // プレイヤーを追加
    tempdata[0] = 0;
    score = 0;      // スコア
    gameover = 0;  // ゲームオーバーカウンタ
    bgy = 0;        // BGオフセット座標
    loopcnt = 0;   // ループカウンタ
    level = 50;    // 難易度
    ppu_enable(1); // 表示を有効化
}

// リザルト画面
void result(void)
{
    char tmpstr[6];
    ppu_enable(0);    // 表示を無効化
    bg_cls();         // BGを全消去
    sp_cls();         // スプライトを全消去
    bg_printstr(11,5,(char *)strgover);  // GAME OVER
    // スコア表示
    bg_printstr(10,10,(char *)strscore);
    num_to_str(tmpstr,score,4);
    bg_printstr(16,10,tmpstr);
    // 難易度表示
    bg_printstr(10,15,(char *)strlevel);
    num_to_str(tmpstr,level,3);
    bg_printstr(16,15,tmpstr);
    pushbutton();    // ボタンの入力待ち
}

// スプライト衝突判定(番号,パレット下限値,しきい値)
// *指定したパレット以上のスプライトにだけ反応
char sp_search(char num,char pal,unsigned char diff)
{
    unsigned char *ptr;
    char ret,tempx,tempy,x,y;
    ptr = (unsigned char *)SPWADDR+(4*num);
    x = ptr[3];     // X座標
    y = ptr[0];     // Y座標
```

```
    ptr = (unsigned char *)SPWADDR;
    for(ret=0; ret<32; ret++,ptr+=4){
        if(ret == num)continue;
        tempy = ptr[0];     // Y座標
        if(tempy == OUTSIDE)continue;
        if((ptr[2] & 3) < pal)continue;
        tempy += diff-y;
        if(tempy > diff*2)continue;  // 衝突なし
        tempx = ptr[3];     // X座標
        tempx += diff-x;
        if(tempx > diff*2)continue;  // 衝突なし
        break; // 衝突
    }
    return(ret);  // スプライト番号を返す
}

// プレイヤー移動処理
void player(char num,unsigned char x,unsigned char y)
{
    unsigned char btn;     // ボタン情報
    btn = controller1();   // コントローラー1入力
    if(btn & BUTTON_LEFT ){
        if(x > 8) x -= 2;     // 左
    }else if(btn & BUTTON_RIGHT){
        if(x < 232) x += 2;   // 右
    }
    if(btn & BUTTON_UP ){
        if(y > 32) y -= 2;    // 上
    }else if(btn & BUTTON_DOWN){
        if(y < 208) y += 2;   // 下
    }
    if((btn & BUTTON_A)&&(tempdata[0] >= 32)){  // Aボタン
        tempdata[0] = 0;
        sp_append(x,y-8,TILE_MYSHOT,ATR_FLIPV+PAL_1);
    }
    if(tempdata[0] < 64) tempdata[0]++;
    sp_ofs(num,x,y);
    // 当たり判定は4フレームに1回だけ
    if(loopcnt & 3)return;
    if(gameover==0){
        if(sp_search(0,2,10)< 32){  // 衝突
            pulse1(250);     // 矩形波1サウンド再生
            gameover = 1;    // ゲームオーバー
            sp_chr(num,TILE_BAKUHA,PAL_1); // 爆発
            tempdata[num]=0;
        }
    }
}
```

```
// 自機の弾の移動処理
void myshot(char num,unsigned char x,unsigned char y)
{
    char ret;
    if(y<8){
        y = OUTSIDE;  // 画面外へ
        sp_ofs(num,x,y);  // スプライトの移動
        return;
    }
    y -= 8;
    // 当たり判定は2フレームに1回だけ
    if(loopcnt & 1){
        ret = sp_search(num,3,15);
        if(ret < 32){  // 衝突
            sp_chr(ret,TILE_BAKUHA,PAL_1); // 爆発
            tempdata[ret] = 0;
            pulse1(500);  // 矩形波1サウンド再生
            score++;      // スコアを加算
            y = OUTSIDE;  // 画面外へ
        }
    }
    sp_ofs(num,x,y);  // スプライトの移動
}

// 爆発処理
void bakuhatu(char num,unsigned char x,unsigned char y)
{
    tempdata[num]++;
    if(tempdata[num] > 30){  // 一定時間経過
        y = OUTSIDE;
        sp_ofs(num,x,y);
    }
}

// 敵機の弾
void beam(char num,unsigned char x,unsigned char y)
{
    char x1,y1;
    x1 = (tempdata[num] >> 4)-8;
    y1 = (tempdata[num] & 0xf)-8;
    x += x1;
    y += y1;
    if((x<8)||(x>240)||(y<8)){
        y = OUTSIDE;
    }
    sp_ofs(num,x,y);
}
```

```
// 符号
int fnc_sgn(int a)
{
    if(a < 0)
        return(-1);

    return(1);
}

// 絶対値
int fnc_abs(int a)
{
    if(a < 0)
        return(-a);

    return(a);
}

// 敵の機体の移動処理
void enemy(char num,unsigned char x,unsigned char y)
{
#define SPEED 2   // 弾の速度
    char num2;
    int delta_x,delta_y;
    if(tempdata[num] & 0x40){
        x+=1;
    }else{
        x-=1;
    }
    y += ((tempdata[num] >> 4) & 3);
    if((x<8)||(x>240)){
        y = OUTSIDE;
    }
    sp_ofs(num,x,y);
    tempdata[num]++;
    if((tempdata[num] & 0x3f)!=0)return;
    if(gameover)return;
    if((random() & 0xff) > level)return;
    delta_x = sp_getx(0) - x;
    delta_y = sp_gety(0) - y;
    if((delta_x==0)&&(delta_y==0))return;
    num2 = sp_append(x,y,TILE_BEAM,PAL_2); // 敵の弾
    if(num>=32)return;
    if(fnc_abs(delta_x) > fnc_abs(delta_y)){
        delta_y = delta_y * SPEED / fnc_abs(delta_x);
        delta_x = SPEED * fnc_sgn(delta_x);
    }else{
```

```
        delta_x = delta_x * SPEED / fnc_abs(delta_y);
        delta_y = SPEED * fnc_sgn(delta_y);
    }
    delta_x += 8;
    delta_y += 8;
    tempdata[num2] = ((char)delta_x << 4)+(char)delta_y;
}

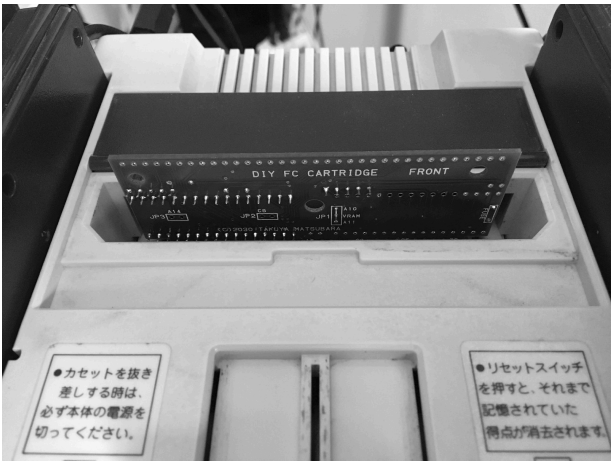
// メイン関数
char NesMain(void)
{
    unsigned char i,x,y;
    fc_init();      // 全機能の初期化
    // パターンテーブル設定 & カラーパレット設定
    ppu_pattern((unsigned char *)pattern,0,256);
    ppu_pattern8x16((unsigned char *)pattern+4096,256,256);
    ppu_palette((char *)palette,PAL_0+4,4);
    while (1){
       title();   // タイトル＆ゲーム開始
        while (gameover < 180){
            for(i=0; i<32; i++){
                y = sp_gety(i);
                if(y==OUTSIDE)continue; // 画面外
                x = sp_getx(i);
                if(y >= 232){
                    sp_remove(i); // 画面外は消去
                    continue;
                }
                switch(sp_gettile(i)/8){
                case TILE_SHIP/8:
                    player(i,x,y);  // プレイヤー
                    break;
                case TILE_MYSHOT/8:
                    myshot(i,x,y); // 自機の弾
                    break;
                case TILE_ENEMY/8:
                    enemy(i,x,y);  // 敵の機体
                    break;
                case TILE_BEAM/8:
                    beam(i,x,y);   // 敵の弾
                    break;
                case TILE_BAKUHA/8:
                    bakuhatu(i,x,y); // 爆発処理
                    break;
                }
            }
            loopcnt++;
            // 敵の出現
```

```
            if((loopcnt & 0x1f)==0){
                if((random() & 255) <= level){
                    x = (random() % 18)*8 + 60;
                    y = 0;
                    sp_append(x,y,TILE_ENEMY,PAL_3);
                }
                if(level<255)level++;//難易度を上げる
            }
            sp_anime(loopcnt & 31);
            if(gameover)gameover++;
            bgy--;
            if(bgy == 0xff)bgy=0xef;
            waitvsync();          // Vブランク待ち
            sp_dmastart();        // OMA DMA転送開始
            bg_scroll(0,bgy);     // BGスクロール
        }
        result();   // リザルト
    }
    return 0;
}
```

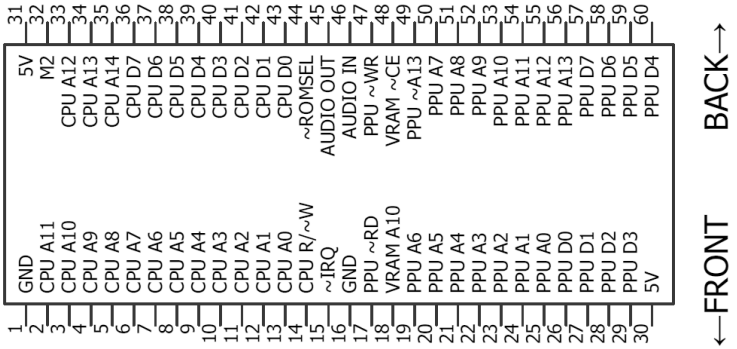
《コラム》シューティングゲームは大変

ファミコンで最も作りにくいゲームは、おそらくシューティングゲームだと思います。キャラを出しすぎると処理落ちしますし、スプライトが消えてしまいます。ハードの性能に限界がありすぎです。過去の名作は超人プログラマーの超絶テクニックが支えています。このゲームの場合、当たり判定の呼び出し回数を減らすことで、できるだけ処理落ちしないように工夫しています。

第8章　実機で動かす



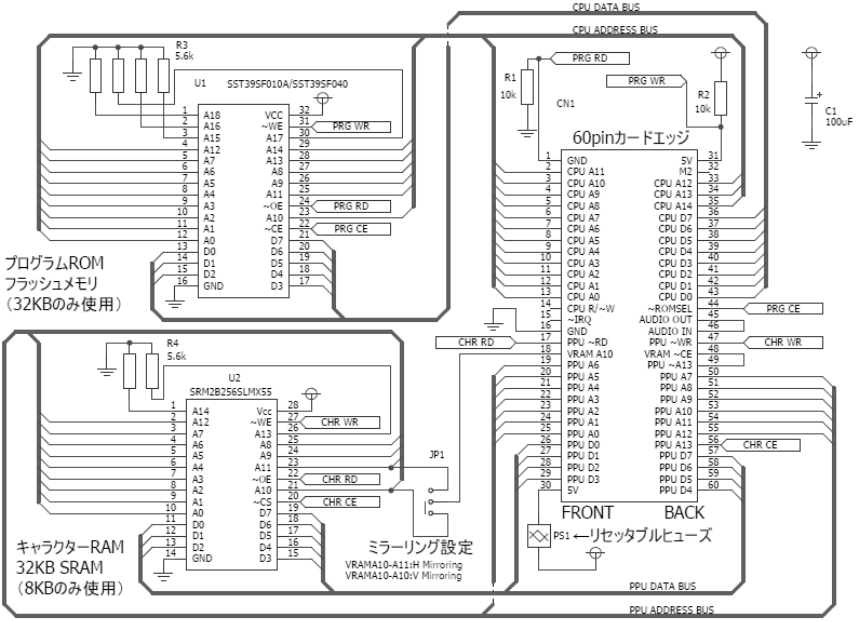
カセットの基板を作って実機のファミコンで遊んでみましょう。



カセット内部に基板があり、外に出ている端子を「カードエッジ」と呼びます。その受け手となるファミコン側のコネクタを「カードエッジコネクタ」と呼びます（取扱説明書では「カセットコネクタ」です）。カードエッジコネクタは全60ピン。ピン間隔は2.54mmです。コネクタの中にはVRAMの

チップイネーブル端子や、サウンドの入出力端子もあります。将来の機能拡張を見据えて設計されています。

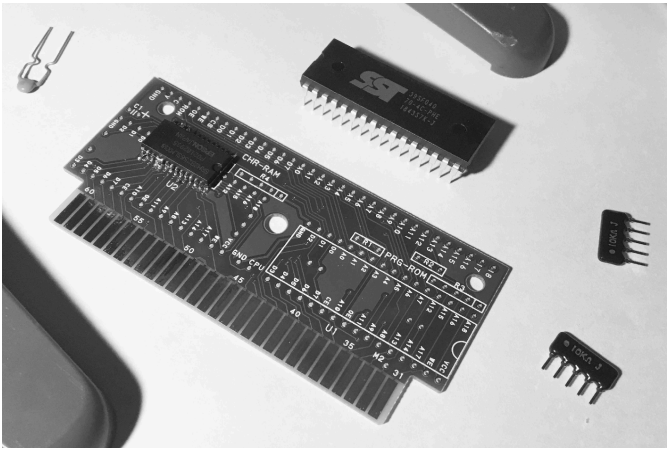
8-1 カセットの自作



カセットの回路図です。ファミコンの妙な仕様ですが、読み込み信号の論理が逆になっていて、書き込み信号と兼用になっています。そこで、読み込み信号と書き込み信号を分離するため、GNDとVCCを勝手に信号用に使っています。行儀の悪いオリジナルの回路です。万が一の事故に備えてリセッタブルヒューズを取り付けておきました。

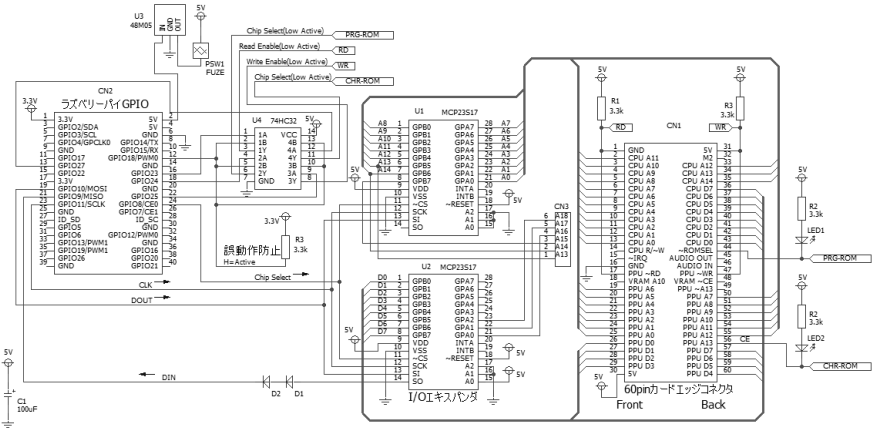
PRG-ROMには動作電圧5Vのフラッシュメモリを使います。フラッシュメモリは「DigiKey」で購入しました。DigiKeyは海外を拠点にする電子

部品の通販サイトです。商品の豊富さでは他を圧倒しています。代金が6000円以上なら送料無料です。フラッシュメモリは128KB(1Mbit)の「SST39SF010A」か、512KB(4Mbit)の「SST39SF040」になりました。もったいないですが、32KBしか使っていません。SRAMは32KBの「SRM2B256SLMX55」を使用しました。8KBしか使っていないので、やはりもったいないです。SRAMは秋月電子通商で買いましたが、残念ながら今は在庫切れです。

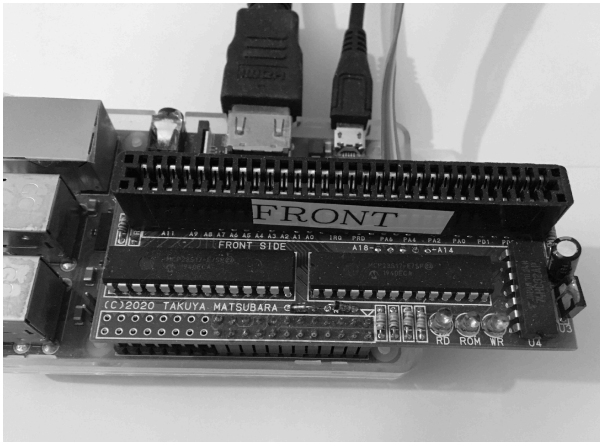


プリント基板の設計は「CADLUS　X」というツールを使いました。基板の寸法は市販のカセットを参考にしました。プリント基板の製造は「P板.com」に注文しました。作る枚数を減らしても、代金は3万円くらいになります。KiCadと海外のプリント基板屋さんを組み合わせたら、もっと安く済むと思います。プリント基板が届いたら、部品をハンダ付けします。

8-2 ライターの自作



フラッシュメモリのライター(書き込み装置)の回路図です。ラズベリーパイ(ラズパイ)に取り付けて使います。フラッシュメモリの動作電圧が5Vなのに対して、ラズパイの電圧は3.3Vなので、電圧を変換しないといけません。ラズパイの5V端子は電圧が不安定なので、三端子レギュレータを取り付けます。三端子レギュレータは低ドロップタイプを使ってください。



完成したフラッシュメモリライターです。この基板も「P板.com」に注文

しました。書き込みツールはPiSTARTER版とPython版を作りました。nesファイルを書き込む時はヘッダー情報の16バイトを削ります。



　ミラーリングの設定を間違えると、このようにBGの画面が欠けて表示されてしまいます。カセット内の配線を正しく直しましょう。

自作カセットと自作ライターの資料をGitHubで公開しています。
https://github.com/nicotakuya/diyfccartridge

「ファミコンゲームの作り方」の専用ページです。掲載プログラムを全て公開しています。
https://github.com/nicotakuya/How-to-Create-FC-Games

参考資料

以下のWebサイトを参考にさせて頂きました。

Enri's Home PAGE
http://cmpslv3.stars.ne.jp/

NesDev.org
https://www.nesdev.org/

コンソールゲーム機研究所　NES研究室
https://hp.vector.co.jp/authors/VA042397/nes/index.html

任天堂 ゲーム専用機販売実績
https://www.nintendo.co.jp/ir/finance/hard_soft/index.html

総務省　標準テレビジョン・データ多重放送及び高精細度テレビジョン・データ多重放送に関する送信の標準方式
https://www.tele.soumu.go.jp/horei/reiki_honbun/72335000001.html

初出：

「Jump Game」

日経ソフトウエア2020年11月号、2021年1月号、2021年3月号

書名：　ファミコンゲームの作り方

著者名：　松原 拓也

発行日：　2026年10月24日 初版
